



# 7장 군집화

1팀 김예나, 유연, 장인서

# 목차

---

#01 7.0 군집화 개념 ~ 7.2 군집 평가

#02 7.3 평균 이동 ~ 7.5 DBSCAN

#03 (추가) 병합 군집, 계층적 군집 ~ 7.6 군집화 실습



# 7-0. 군집화 개념

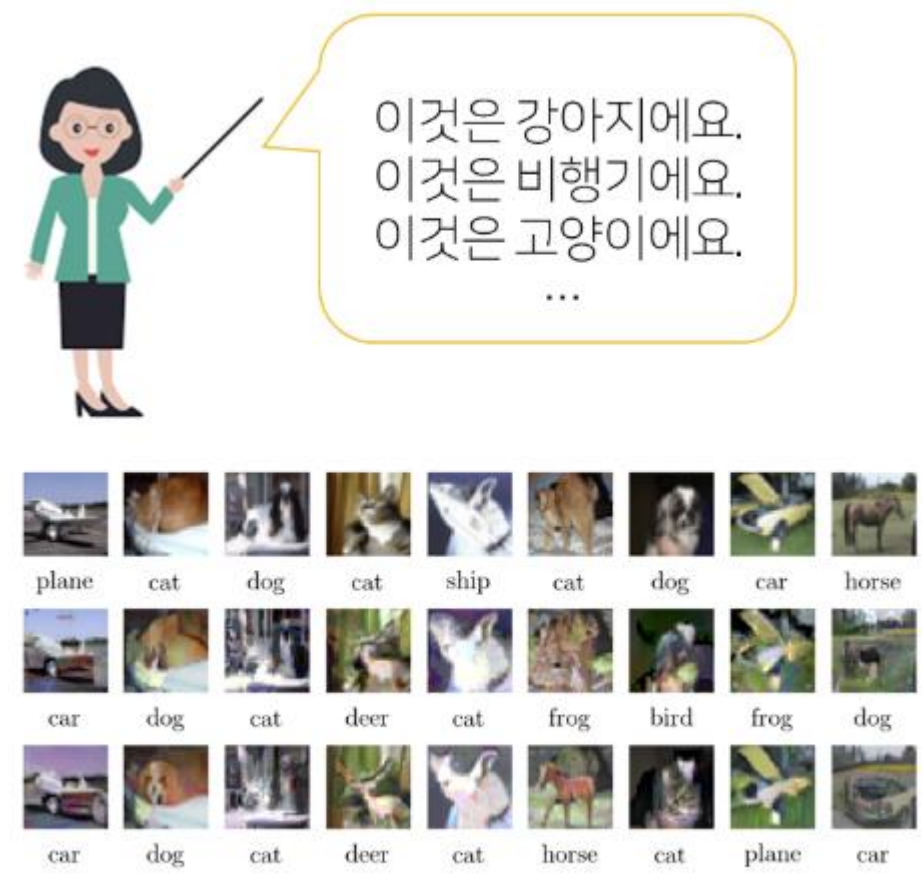


# #7-0. 군집화 개념

## 지도 학습 VS 비지도 학습

지도학습 (Supervised Learning) 이란?

- 정답이 있는 데이터를 활용해 데이터를 학습시키는 것
- 각각의 데이터에 정답을 작성하는 것 :  
'레이블(label)을 달아준다



지도학습 - 지도자가 있는 학습방법

비지도학습 (Unsupervised Learning)이란?

- 대량의 데이터에 레이블을 달지 않고, 패턴을 찾게 하는 것
- 패턴을 찾아서 유사한 데이터끼리 묶어내는 과정 : 클러스터링, 군집



지도자가 없는 학습방법 - 비지도학습

# #7-0. 군집화 개념

## 군집화란?

주어진 정답 값이 없는 데이터들에서 개체들을 비슷한 것끼리 그룹을 나누는 것 . (비슷하다 = 유사도 = 거리)

## #비계층적 군집화

- 거리 기반 군집화 : 각 데이터 포인트들 간의 거리를 기반으로 군집을 형성

Ex) K-means

- 밀도 기반 군집화 : 데이터가 밀집된 지역을 중심으로 클러스터를 형성

EX) DBSCAN

## #계층적 군집화

: 가까운 개체끼리 묶거나 멀리 떨어진 개체를 분리하여 군집을 만드는 것

- 합병형(병합형) : 가장 가까운 두 개를 하나의 그룹으로 묶는것을 반복하여 하나의 군집을 만드는 것

Ex) 최단 연결법, 평균 연결법, 와드 연결법, 중심 연결법, 완전 연결법 등

- 분할적 : 큰 군집에서 작은 군집으로 분할하는 방식



# #7-0. 군집화 개념

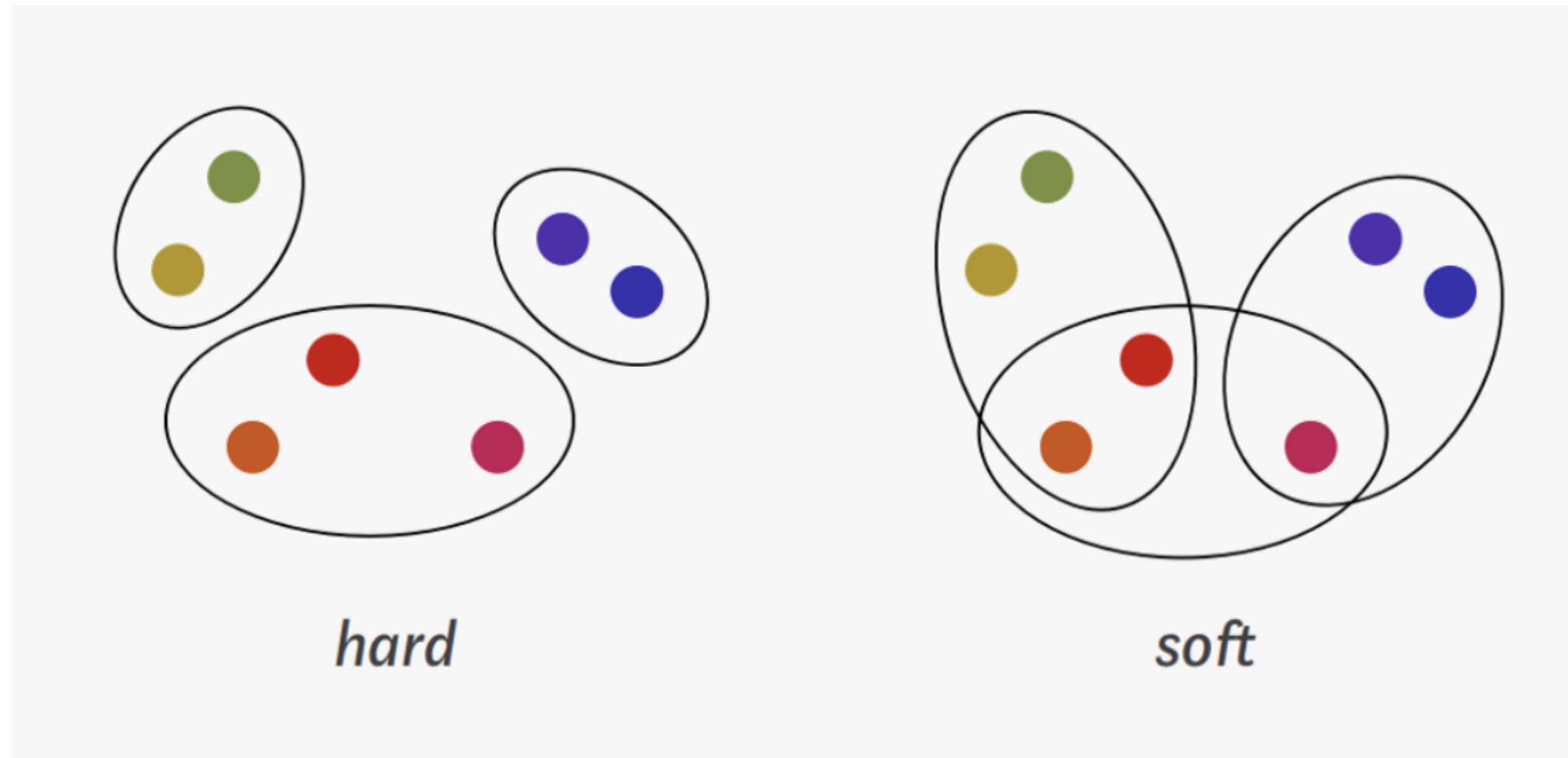
## # 하드 군집 VS 소프트 군집

#하드 군집(하드 클러스터링)

- 하나의 데이터가 정확히 하나의 군집에 할당하는 것
- EX) hierarchical clustering, k-means, dbscan, optics

#소프트 군집(소프트 클러스터링)

- : 하나의 데이터가 다수의 군집에 할당하는 것
- topic models, fcm, soft k-means



# 7-1. K-means



# #7-1. K-means

# K-means(K-평균) : 대표적인 거리 기반 알고리즘으로, **군집 중심점(centroid)**이라는 특정한 임의의 지점을 선택해 해당 중심에 가장 가까운 포인트들을 선택하는 군집화 기법

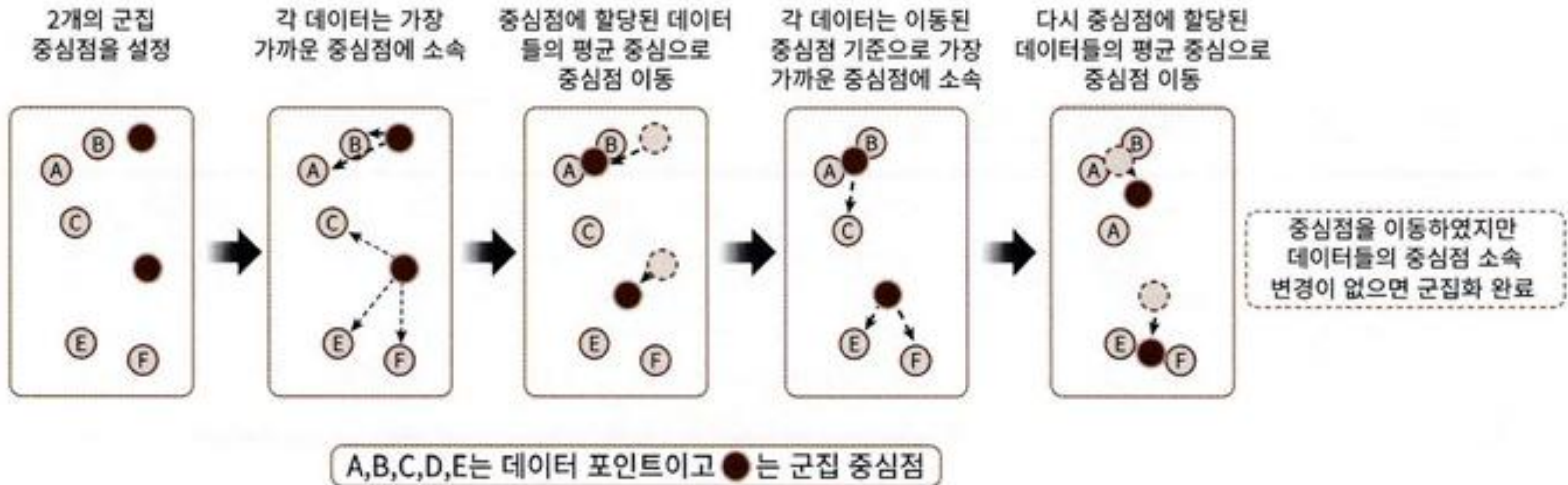
- > 군집 중심점은 선택된 포인트의 평균 지점으로 이동
- > 이동된 중심점에서 다시 가까운 포인트를 선택
- > 다시 중심점을 평균 지점으로 이동

해당 프로세스를 반복적으로 수행하며, 모든 데이터 포인트에서 더 이상 중심점의 이동이 없을 경우에 반복을 멈추고 해당 중심점에 속하는 데이터 포인트들을 군집화한다.



# #7-1. K-means

## # K-means 알고리즘의 동작 방식



# #7-1. K-means

## # K-means의 장점

- 알고리즘이 쉽고 간결
- 일반적인 군집화에서 가장 많이 활용됨

## # K-means의 단점

- 거리 기반 알고리즘으로, 속성의 개수가 매우 많은 경우 군집화 정확도가 떨어짐
- 반복을 수행하는데, 반복 횟수가 많을 경우 수행 시간이 매우 느려짐
- 몇 개의 군집(cluster)을 선택해야 할지 가이드하기 어려움

# #7-1. K-means

#사이킷런 K-means 클래스

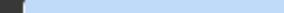
```
class sklearn.cluster.KMeans(n_clusters=8, init='k-means++', n_init=10, max_iter=300, tol=0.0001,
                              precompute_distances='auto', verbose=0, random_state=None,
                              copy_x=True, n_jobs=1, algorithm='auto')
```

| 주요 파라미터         | 역할                                                                        |
|-----------------|---------------------------------------------------------------------------|
| N_clusters      | 군집화할 개수, 즉 군집 중심점의 개수                                                     |
| Init            | 초기에 군집 중심점의 좌표를 설정할 방식<br>보통 임의로 중심을 설정하지 않고, 일반적으로 k-means++ 방식으로 최초 설정함 |
| Max_iter        | 최대 반복 횟수<br>이 횟수 이전에 모든 데이터의 중심점 이동이 없으면 종료                               |
| Cluster_centers | 각 군집 중심점 좌표                                                               |

EWCHA  
EUROPEAN

-> 붓꽃 데이터 로드, dataframe으로 변경

|   | sepal_length | sepal_width | petal_length | petal_width |
|---|--------------|-------------|--------------|-------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         |



The screenshot shows a Jupyter Notebook cell with the following content:

```
KMeans
```

```
KMeans(n_clusters=3, random_state=0)
```

→ Labels\_값이 0,1,2로 되어있음 : 각 레코드가 첫 번째 군집, 두 번째 군집, 세 번째 군집에 속함

# #7-1. K-means

## # K-means을 이용한 붓꽃 데이터 세트 군집화

```
[5] 1 irisDF['target'] = iris.target
    2 irisDF['cluster']=kmeans.labels_
    3 iris_result = irisDF.groupby(['target','cluster'])['sepal_length'].count()
    4 print(iris_result)
```

```
target  cluster
0        1      50
1        0      47
        2       3
2        0      14
        2      36
Name: sepal_length, dtype: int64
```

-> 실제 분류값인 target과 군집화 분류값인 cluster 레벨로 적용해 target과 cluster 값 개수를 비교

[결과]

분류 타깃이 0값인 데이터 -> 1번 군집으로 모두 잘 그룹핑  
Target 1 -> 2개만 2번 군집, 48개는 0번 군집으로 그룹핑  
Target 2 -> 14개가 0번 군집, 36개는 2번 군집으로 분산

```
1 from sklearn.decomposition import PCA
2 pca = PCA(n_components=2)
3 pca_transformed = pca.fit_transform(iris.data)
4 irisDF['pca_x'] = pca_transformed[:, 0]
5 irisDF['pca_y'] = pca_transformed[:, 1]
6 irisDF.head(3)
```

|   | sepal_length | sepal_width | petal_length | petal_width | target | cluster | pca_x     | pca_y     |
|---|--------------|-------------|--------------|-------------|--------|---------|-----------|-----------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | 0      | 1       | -2.684126 | 0.319397  |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | 0      | 1       | -2.714142 | -0.177001 |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | 0      | 1       | -2.888991 | -0.144949 |

- ➔ PCA로 붓꽃 데이터 세트의 4가지 속성을 2개로 차원 축소, X좌표와 Y좌표로 개별 데이터 표현
- ➔ Pca\_x : X 좌표값, pca\_y : Y좌표값

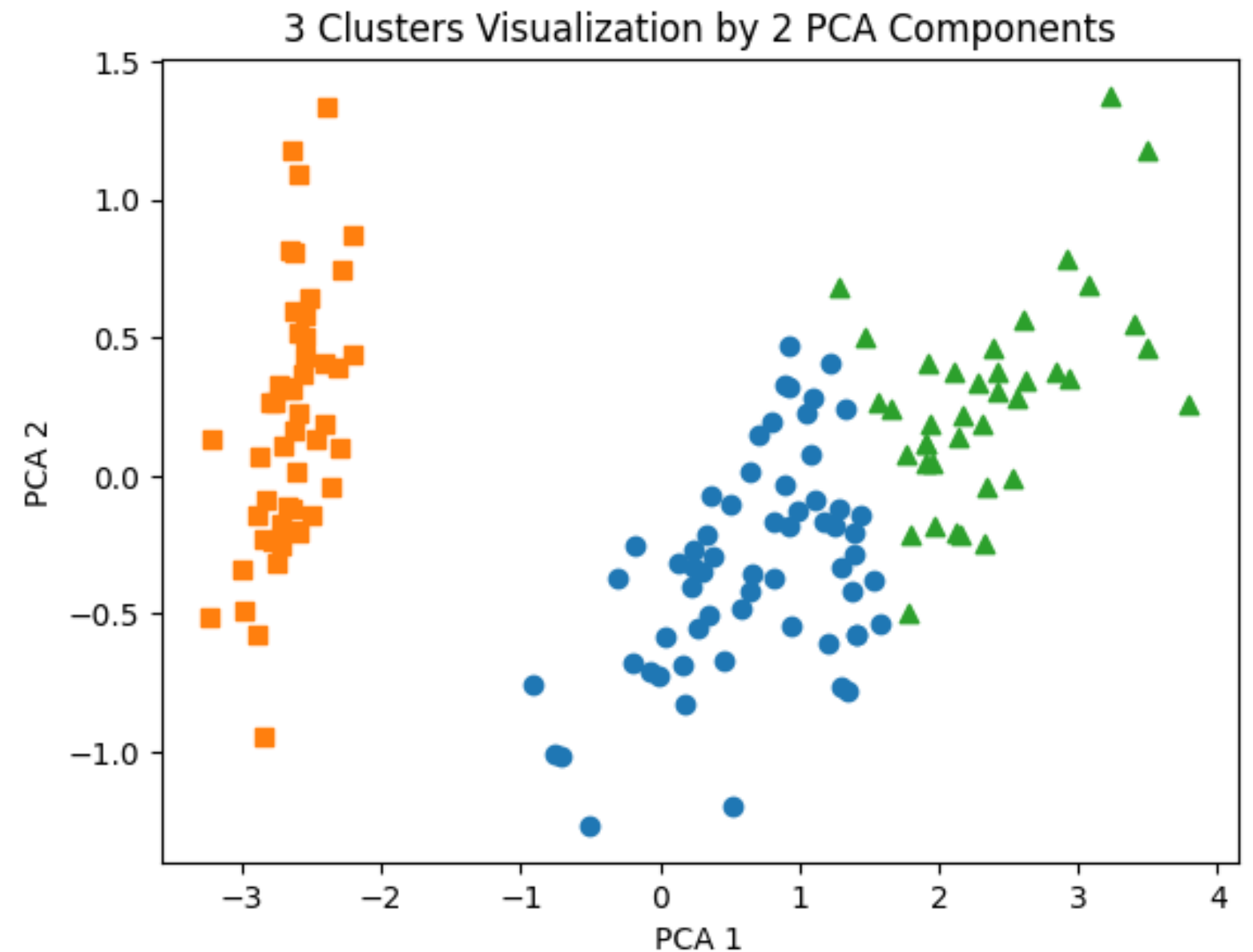


# #7-1. K-means

## # K-means을 이용한 붓꽃 데이터 세트 군집화

```
1 # 군집 값이 0, 1, 2인 경우마다 별도의 인덱스로 추출
2 marker0_ind = irisDF[irisDF['cluster']==0].index
3 marker1_ind = irisDF[irisDF['cluster']==1].index
4 marker2_ind = irisDF[irisDF['cluster']==2].index
5 # 군집 값 0, 1, 2에 해당하는 인덱스로 각 군집 레벨의 pca_x, pca_y 값 추출. o, s, ^로 마커 표시
6 plt.scatter(x=irisDF.loc[marker0_ind, 'pca_x'], y=irisDF.loc[marker0_ind, 'pca_y'], marker='o')
7 plt.scatter(x=irisDF.loc[marker1_ind, 'pca_x'], y=irisDF.loc[marker1_ind, 'pca_y'], marker='s')
8 plt.scatter(x=irisDF.loc[marker2_ind, 'pca_x'], y=irisDF.loc[marker2_ind, 'pca_y'], marker='^')
9 plt.xlabel('PCA 1')
10 plt.ylabel('PCA 2')
11 plt.title('3 Clusters Visualization by 2 PCA Components')
12 plt.show()
```

- cluster 1(네모) : 다른 군집과 잘 분리
- cluster 0(동그라미) & cluster2(세모) : 상당 수준 분리되어 있지만 네모만큼 명확히 분리되어 있지 않음.
- > Cluster 0과 1의 경우 속성의 위치 자체가 명확히 분리되기 어려운 부분이 존재





# #7-1. K-means

## # 군집화 알고리즘 테스트를 위한 데이터 생성

데이터 생성기 API

1) make\_blobs()

: 호출 시 피쳐 데이터 세트와 타깃 데이터 세트가 튜플(Tuple)로 반환됨, 개별 군집의 중심점과 표준 편차 제어 기능

[주요 파라미터]

- n\_samples : 생성할 총 데이터의 개수
- n\_features : 데이터의 피쳐
- Centers : int 값 – 군집 개수, ndarray 형태 – 개별 군집 중심점의 좌표
- Cluster\_std : 생성될 군집 데이터의 표준 편차로, 군집별로 서로 다른 표준 편차를 가진 데이터 세트를 만들 때 사용

2) make\_classification()

: 노이즈를 포함한 데이터를 만드는 데 유용

=>make\_blobs, make\_classification은 큰 차이는 없다!

```
[24] 1 import numpy as np
      2 import matplotlib.pyplot as plt
      3 from sklearn.cluster import KMeans
      4 from sklearn.datasets import make_blobs
      5 %matplotlib inline
      6
      7 X, y = make_blobs(n_samples=200, n_features=2, centers=3, cluster_std=0.8, random_state=0)
      8 print(X.shape, y.shape)
      9 # y target 값의 분포를 확인
     10 unique, counts = np.unique(y, return_counts=True)
     11 print(unique, counts)
```

↻ (200, 2) (200,)  
[0 1 2] [67 67 66]

X, y = make\_blobs(n\_samples=200, n\_features=2, centers=3, random\_state=0)을 호출  
: 총 200개의 레코드와 2개의 피쳐가 3개의 군집화 기반 분포도를 가진 피쳐 데이터 세트 X와, 동시에 3개의 군집화 값을 가진 타깃 데이터 세트 y가 반환됨

# #7-1. K-means

## # 군집화 알고리즘 테스트를 위한 데이터 생성

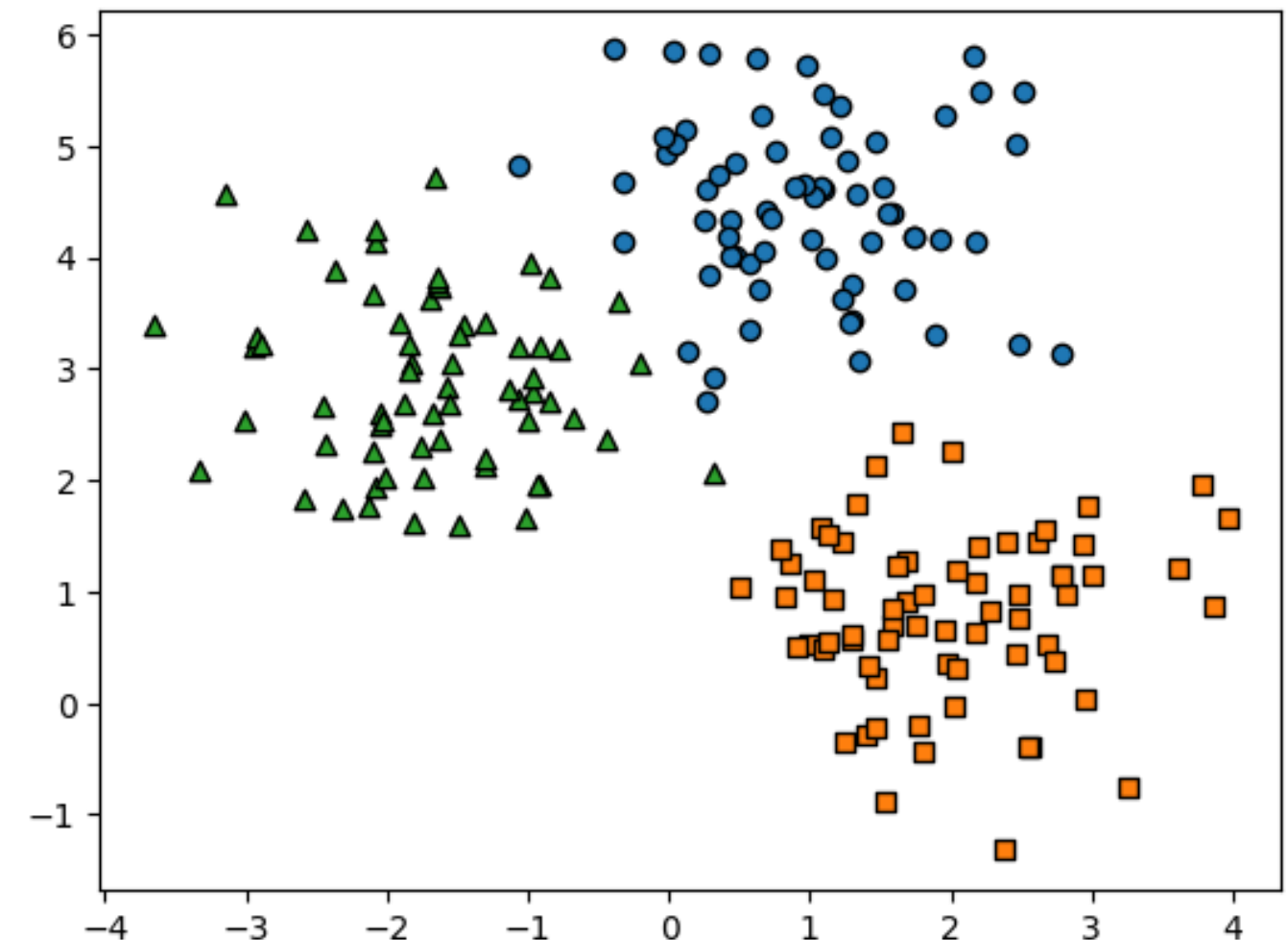
```
[25] 1 import pandas as pd
      2 clusterDF = pd.DataFrame(data=X, columns=['ftr1', 'ftr2'])
      3 clusterDF['target'] = y
      4 clusterDF.head(3)
```

|   | ftr1      | ftr2     | target |
|---|-----------|----------|--------|
| 0 | -1.692427 | 3.622025 | 2      |
| 1 | 0.697940  | 4.428867 | 0      |
| 2 | 1.100228  | 4.606317 | 0      |

```
1 target_list = np.unique(y)
2 # 각 타깃별 산점도의 마커 값.
3 markers=['o', 's', '^', 'P', 'D', 'H', 'x']
4 # 3개의 군집 영역으로 구분한 데이터 세트를 생성했으므로 target_list는 [0, 1, 2]
5 # target=0, target=1, target=2 로 scatter plot을 marker별로 생성
6 for target in target_list:
7     target_cluster = clusterDF[clusterDF['target']==target]
8     plt.scatter(x=target_cluster['ftr1'], y=target_cluster['ftr2'], edgecolor='k', marker=markers[target] )
9 plt.show()
```

데이터 가공을 위해 dataframe으로 변경,  
피쳐명은 ftr1, ftr2

Make\_blob()으로 만든 피쳐 데이터 세트로 타깃값 0, 1, 2에 따라 마커를 다르게 해서 산점도를 그림  
→ 3개의 구분될 수 있는 군집 영역으로 피쳐 데이터 세트가 만들어짐

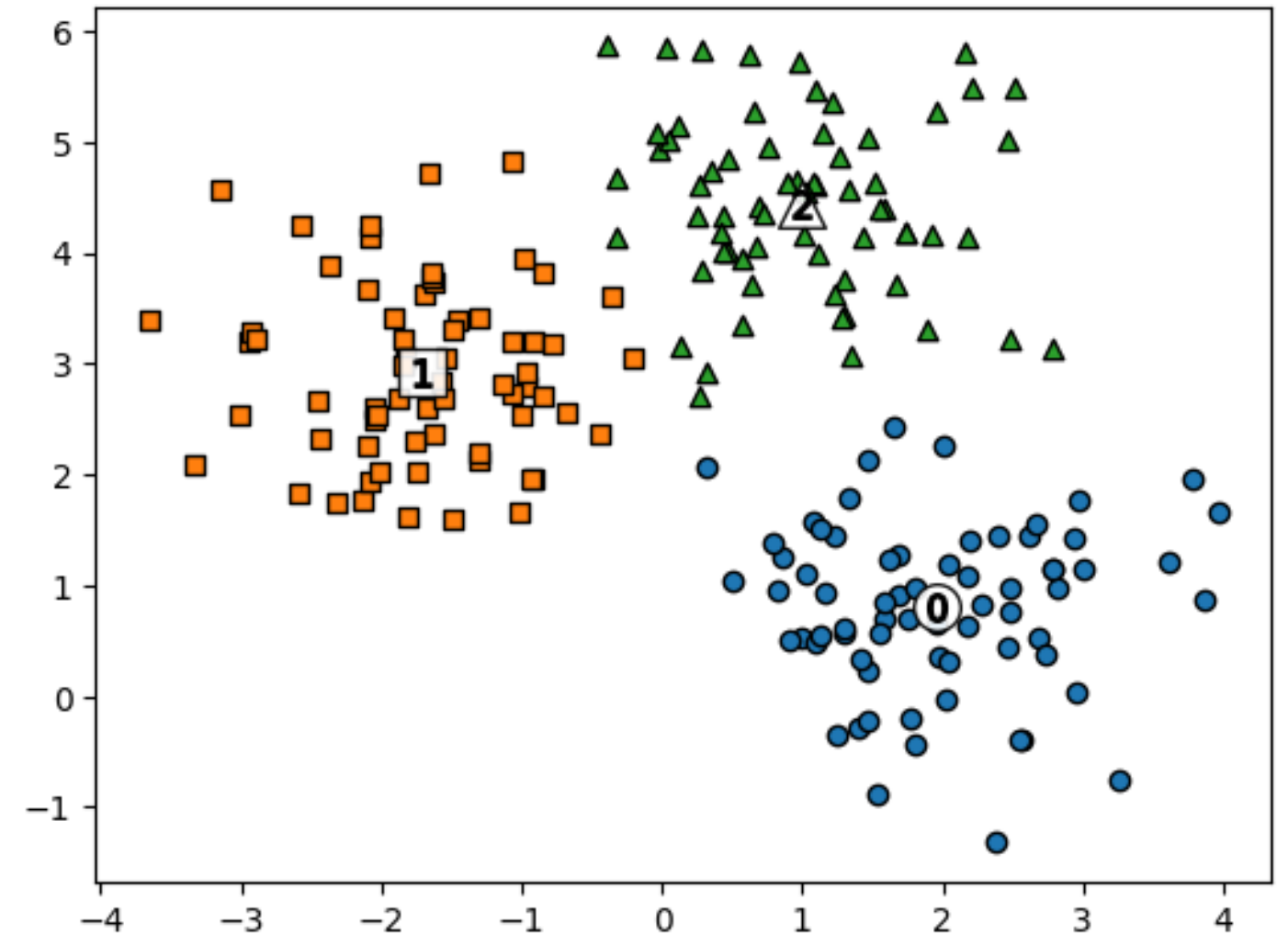




# #7-1. K-means

## # 군집화 알고리즘 테스트를 위한 데이터 생성

```
1 # KMeans 객체를 이용해 X 데이터를 K-Means 클러스터링 수행
2 kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=200, random_state=0)
3 cluster_labels = kmeans.fit_predict(X)
4 clusterDF['kmeans_label'] = cluster_labels
5
6 # cluster_centers_ 는 개별 클러스터의 중심 위치 좌표 시각화를 위해 추출
7 centers = kmeans.cluster_centers_
8 unique_labels = np.unique(cluster_labels)
9 markers=['o', 's', '^', 'P', 'D', 'H', 'x']
10
11 # 군집된 label 유형별로 iteration 하면서 marker 별로 scatter plot 수행.
12 for label in unique_labels:
13     label_cluster = clusterDF[clusterDF['kmeans_label']==label]
14     center_x_y = centers[label]
15     plt.scatter(x=label_cluster['ftr1'], y=label_cluster['ftr2'], edgecolor='k', marker=markers[label])
16 # 군집별 중심 위치 좌표 시각화
17 plt.scatter(x=center_x_y[0], y=center_x_y[1], s=200, color='white', alpha=0.9, edgecolor='k', marker=markers[label])
18 plt.scatter(x=center_x_y[0], y=center_x_y[1], s=70, color='k', edgecolor='k', marker='$_d$' % label)
19
20 plt.show()
```



KMeans 군집화를 수행한 뒤 군집별로 시각화

1) KMeans 객체에 fit\_predict(X)를 수행해 make\_blobs()의 피쳐 데이터 세트인 X 데이터를 군집화 -> clusterDF DataFrame의 'kmeans\_label' 칼럼으로 저장

2) KMeans 객체의 cluster\_centers\_ 속성은 개별 군집의 중심 위치 좌표를 나타내기 위해 사용 : make\_blobs()의 타깃과 kmeans\_label은 군집 번호를 의미하므로 서로 다른 값으로 매핑될 수 있음

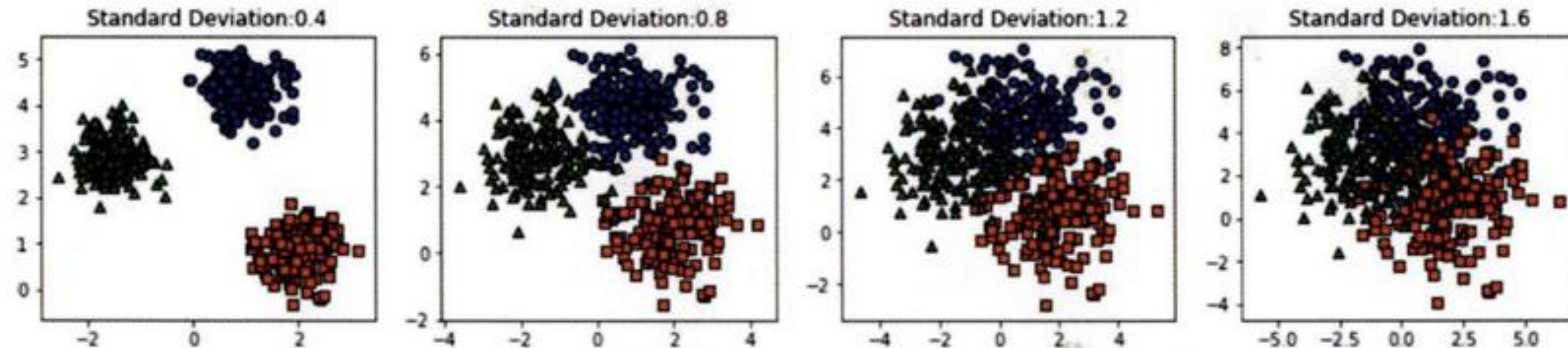
# #7-1. K-means

## # 군집화 알고리즘 테스트를 위한 데이터 생성

```
[12] 1 print(clusterDF.groupby('target')['kmeans_label'].value_counts())
```

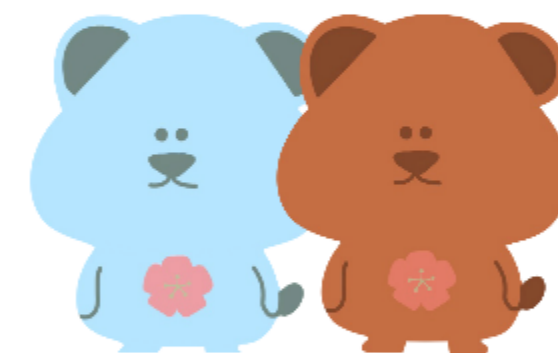
```
target  kmeans_label  count
0       2             66
        1             1
1       0             67
2       1             65
        0              1
Name: count, dtype: int64
```

Target 0] cluster label 0으로, target 1] label 2로, target 2가 label 1 로 거의 대부분 잘 매핑됨



- ➔ make\_blobs()은 cluster\_std 파라미터로 데이터의 분포도를 조절
- ➔ cluster\_std가 작을수록 군집 중심에 데이터가 모여 있으며, 클수록 데이터가 퍼져 있음

## 7-2. 군집 평가



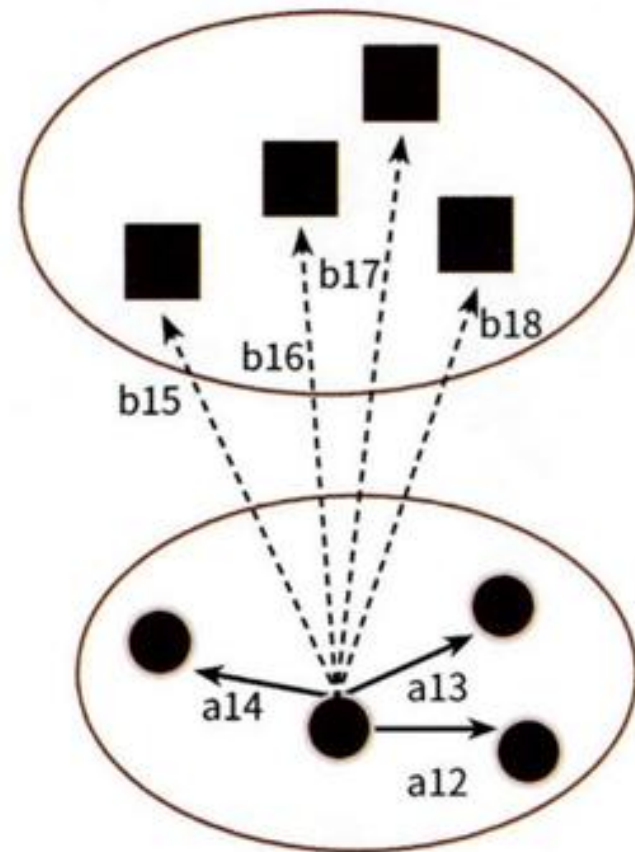
# #7-2. 군집 평가

#실루엣 분석 (silhouette analysis)

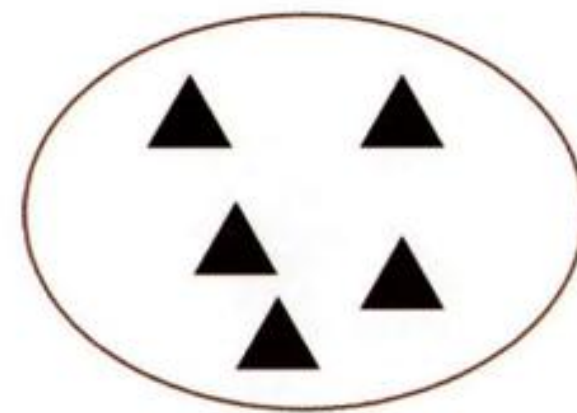
: 각 군집 간의 거리가 얼마나 효율적으로 분리돼 있는지 나타내는 군집 평가 지표  
(= 다른 군집과의 거리는 멀고, 동일 군집끼리는 가깝다)

- 실루엣 계수(silhouette coefficient) : 개별 데이터가 가지는 군집화 지표 -> 해당 데이터가 같은 군집 내의 데이터와 얼마나 가깝게 군집화돼 있고, 다른 군집에 있는 데이터와는 얼마나 멀리 분리돼 있는지를 나타내는 지표

Cluster B  
(Cluster A의 1번 데이터에서 가장 가까운 타 클러스터)



Cluster C



- $a_{ij}$ 는  $i$ 번째 데이터에서 자신이 속한 클러스터내의 다른 데이터 포인트까지의 거리. 즉  $a_{12}$ 는 1번 데이터에서 2번 데이터까지의 거리
- $a(i)$ 는  $i$ 번째 데이터에서 자신이 속한 클러스터내의 다른 데이터 포인트들의 평균 거리. 즉  $a(i) = \text{평균}(a_{12}, a_{13}, a_{14})$
- $b(i)$ 는  $i$ 번째 데이터에서 가장 가까운 타 클러스터내의 다른 데이터 포인트들의 평균 거리. 즉  $b(i) = \text{평균}(b_{15}, b_{16}, b_{17}, b_{18})$

# #7-2. 군집 평가

#i번째 데이터 포인트의 실루엣 계수

$a(i)$  : 해당 데이터 포인트와 같은 군집 내에 있는 다른 데이터 포인트와의 거리를 평균한 값

$b(i)$  : 해당 데이터 포인트가 속하지 않은 군집 중 가장 가까운 군집과의 평균 거리

$\Rightarrow b(i) - a(i)$  = 두 군집 간의 거리가 얼마나 떨어져 있는가

$\Rightarrow$  정규화 :  $\text{MAX}(a(i), b(i))$  값으로 나눔

$$s(i) = \frac{(b(i) - a(i))}{(\max(a(i), b(i)))}$$

-1~1사이의 값

1로 가까워질수록 근처의 군집과 더 멀리 떨어져 있음, 0에 가까울수록 근처 군집과 가까워짐

- 값 : 아예 다른 군집에 데이터 포인트가 할당

#실루엣 분석 메서드

- `sklearn.metrics.silhouette_samples(X, labels, metric= 'euclidean**kwargs)` : 각 데이터 포인트의 실루엣 계수 반환
- `sklearn.metrics.silhouette_score(X, labels, metric= 'euclidean', sample_size=None, **kwargs)` : 전체 데이터의 실루엣 계수 값을 평균해 반환 -> 일반적으로 이 값이 높을수록 군집화가 잘 됐다고 판단, but 무조건 X



# #7-2. 군집 평가

## # 좋은 군집화의 기준

- 1) 전체 실루엣 계수의 평균값, 즉 사이킷런의 `silhouette_score()` 값  $\Rightarrow$  0 ~ 1 사이의 값을 가지며, 1에 가까울수록 좋다.
- 2) 개별 군집의 평균값의 편차가 크지 않아야 한다.  $\Rightarrow$  개별 군집의 실루엣 계수 평균값이 전체 실루엣 계수의 평균값에서 크게 벗어나지 않는 것이 중요. 만약 전체 실루엣 계수 의 평균값은 높지만, 특정 군집의 실루엣 계수 평균값만 유난히 높고 다른 군집들의 실루엣 계수 평균값은 낮으면 좋은 군집화 조건이 아님

# #7-2. 군집 평가

## # 붓꽃 데이터 세트를 이용한 군집 평가

```
1 from sklearn.preprocessing import scale
2 from sklearn.datasets import load_iris
3 from sklearn.cluster import KMeans
4 # 실루엣 분석 평가 지표 값을 구하기 위한 API 추가
5 from sklearn.metrics import silhouette_samples, silhouette_score
6 import matplotlib.pyplot as plt
7 import numpy as np
8 import pandas as pd
9 %matplotlib inline
10 iris = load_iris()
11 feature_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
12 irisDF = pd.DataFrame(data=iris.data, columns=feature_names)
13 kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=300, random_state=0).fit(irisDF)
14 irisDF['cluster'] = kmeans.labels_
15 # iris의 모든 개별 데이터에 실루엣 계수 값을 구함
16 score_samples = silhouette_samples(iris.data, irisDF['cluster'])
17 print('silhouette_samples( ) return 값의 shape', score_samples.shape)
18 # irisDF에 실루엣 계수 칼럼 추가
19 irisDF['silhouette_coeff'] = score_samples
20 # 모든 데이터의 평균 실루엣 계수 값을 구함.
21 average_score = silhouette_score(iris.data, irisDF['cluster'])
22 print('붓꽃 데이터 세트 Silhouette Analysis Score: {0:.3f}'.format(average_score))
23 irisDF.head(3)
```

silhouette\_samples( ) return 값의 shape (150,)  
붓꽃 데이터 세트 Silhouette Analysis Score: 0.551

|   | sepal_length | sepal_width | petal_length | petal_width | cluster | silhouette_coeff |
|---|--------------|-------------|--------------|-------------|---------|------------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | 1       | 0.852582         |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | 1       | 0.814916         |

- 붓꽃 데이터 세트의 평균 실루엣 계수 값 : 0.553
- 첫 3 row는 1번 군집, 평균적으로 0.8 정도의 높은 실루엣 계수 값 <-> 다른 군집은 낮음

```
1 irisDF.groupby('cluster')['silhouette_coeff'].mean()
```

| silhouette_coeff |          |
|------------------|----------|
| cluster          |          |
| 0                | 0.422323 |
| 1                | 0.797604 |
| 2                | 0.436842 |

dtype: float64

- 0번 군집의 실루엣 계수 평균 : 0.41
  - 1번 군집의 실루엣 계수 평균 : 0.79
  - 2번 군집의 실루엣 계수 평균 : 0.45
- => 0,2번은 1번에 비해 상대적으로 낮음

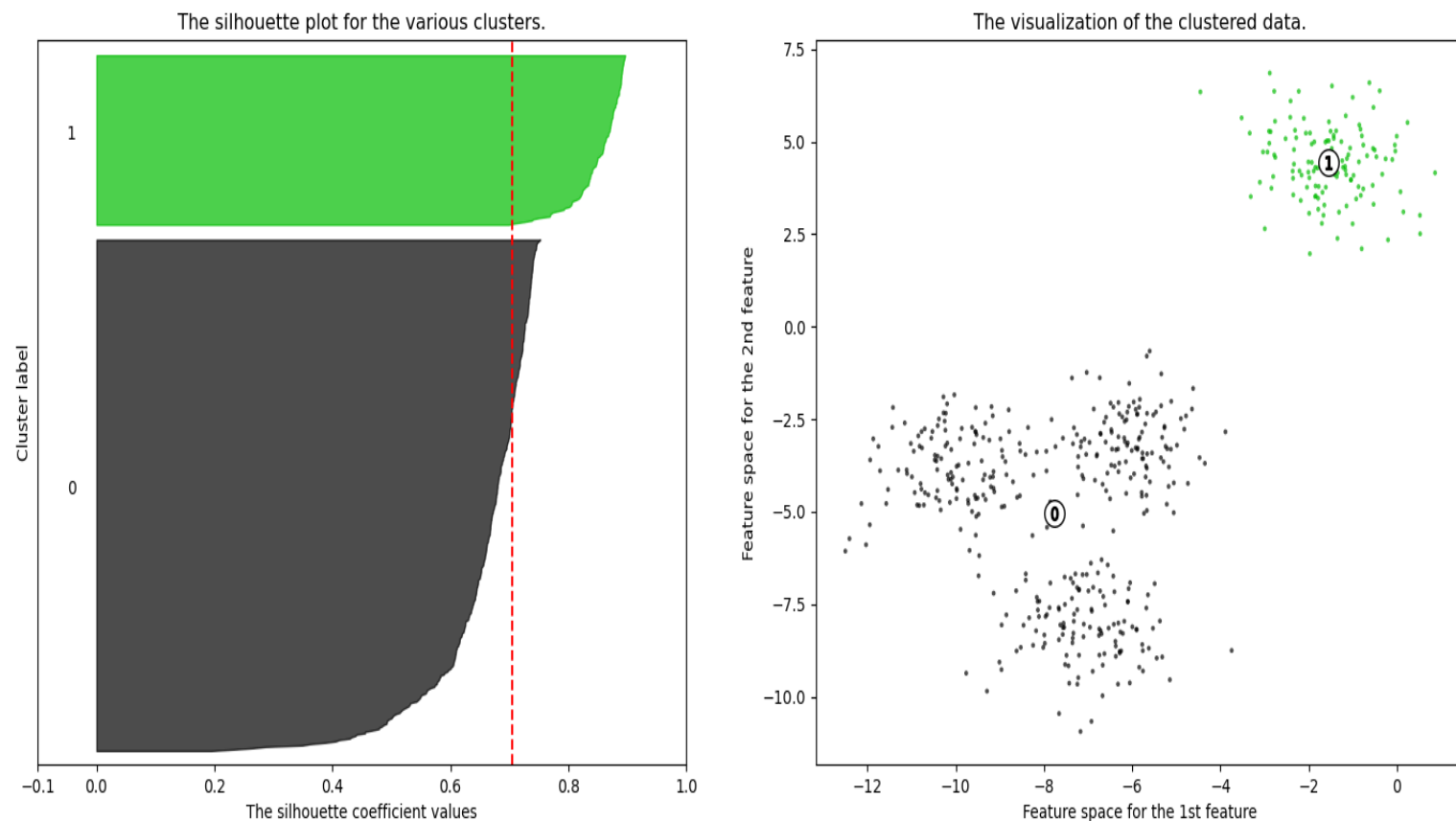


# #7-2. 군집 평가

# 군집별 평균 실루엣 계수의 시각화를 통한 군집 개수 최적화 방법

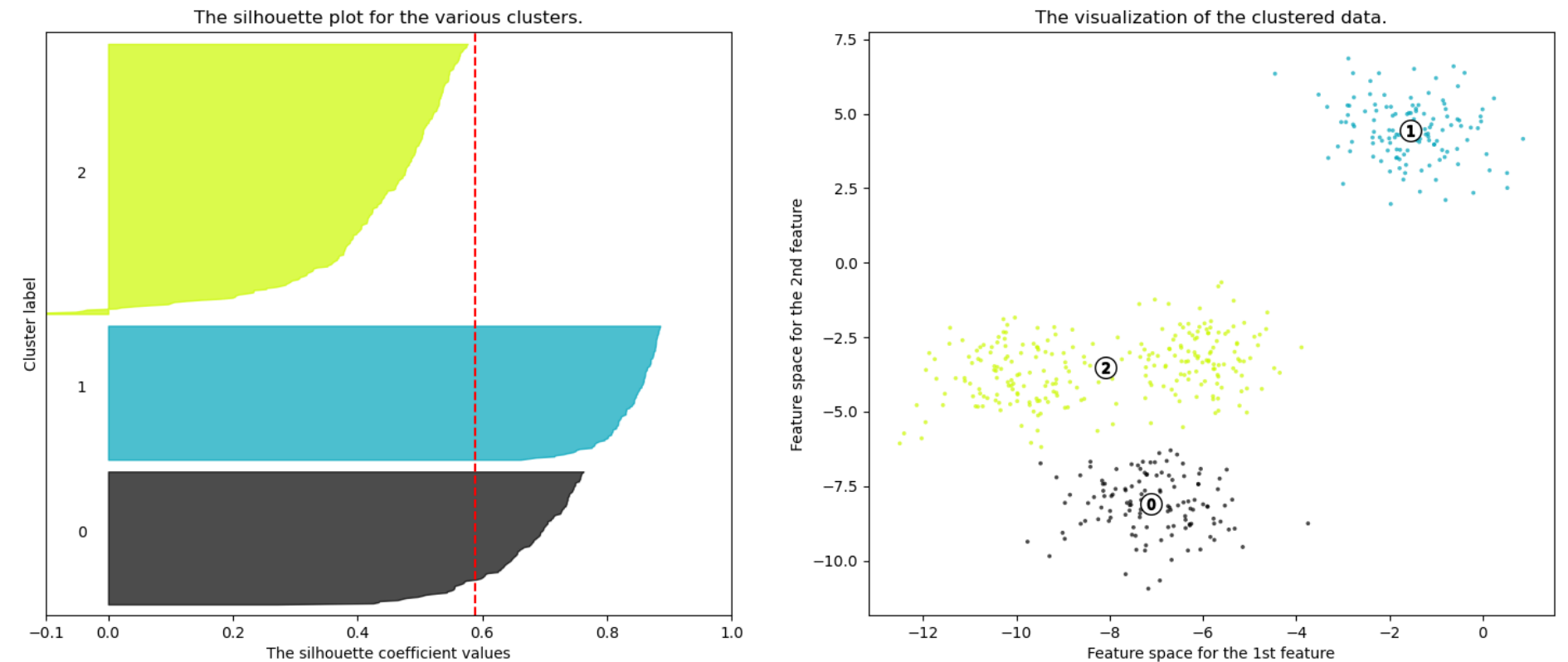
개별 군집별로 적당히 분리된 거리를 유지하면서도 군집 내의 데이터가 서로 뭉쳐 있는 경우에 K-평균 의 적절한 군집개수가 설정됐다고 판단

Silhouette analysis for KMeans clustering on sample data with n\_clusters = 2



- ➔ 군집 개수 2개
- ➔ 전체 데이터 평균 실루엣 계수 : 0.704
- ➔ 1번 군집과 0번 군집은 떨어져 있음
- ➔ 0번 군집은 내부 데이터끼리 많이 떨어져 있음

Silhouette analysis for KMeans clustering on sample data with n\_clusters = 3

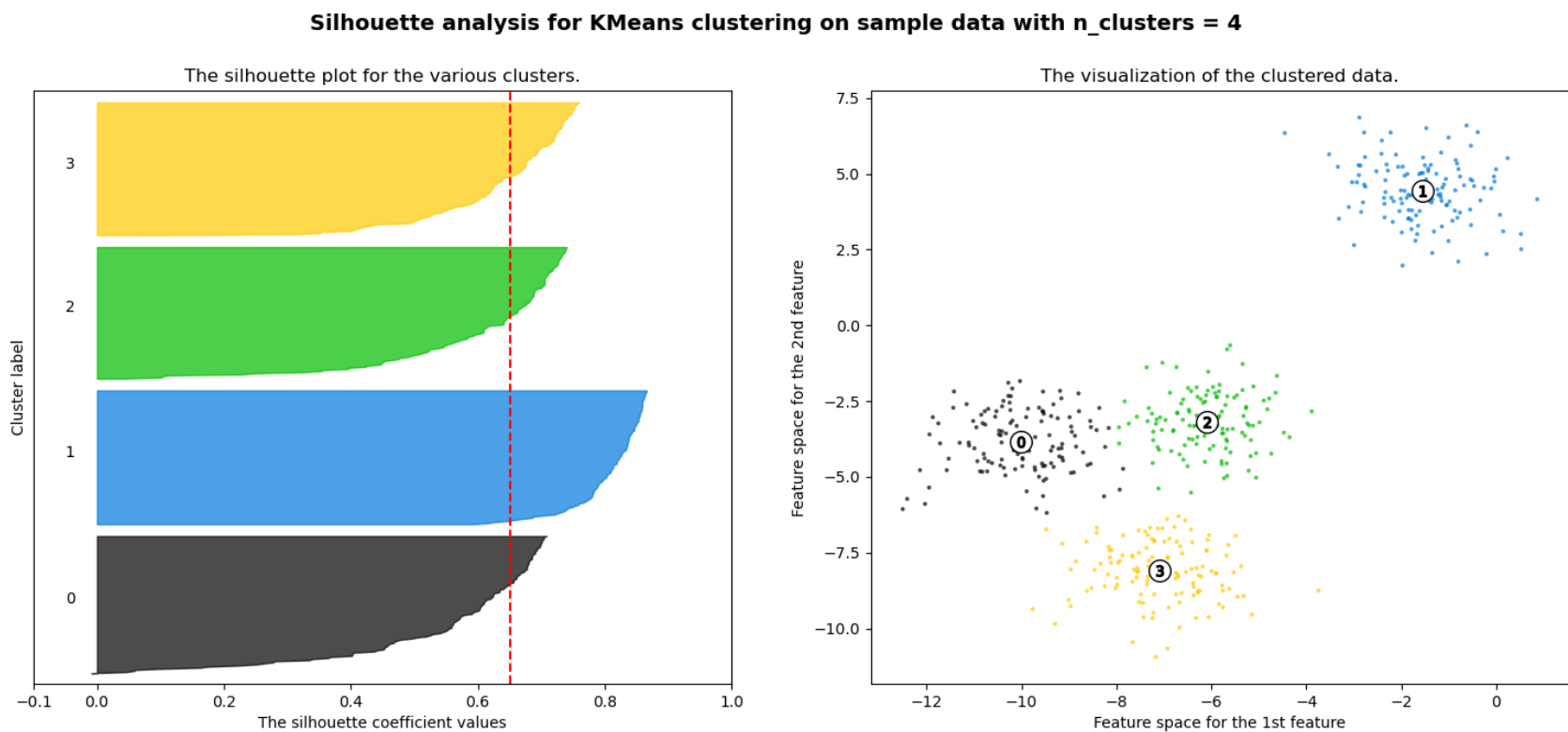


- ➔ 군집 개수 3개
- ➔ 전체 데이터 평균 실루엣 계수 : 0.588
- ➔ 0번의 경우 내부 데이터 간 거리가 멀고, 2번 군집과 가까우며, 평균보다 낮은 실루엣 계수

# #7-2. 군집 평가

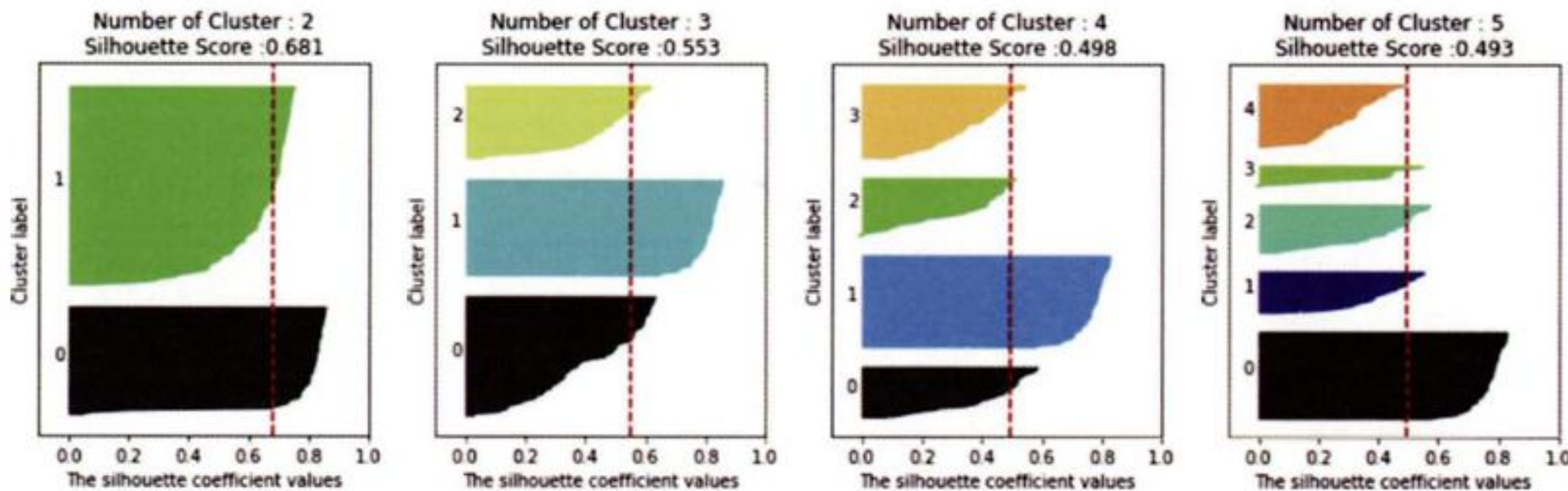
## # 군집별 평균 실루엣 계수의 시각화를 통한 군집 개수 최적화 방법

개별 군집별로 적당히 분리된 거리를 유지하면서도 군집 내의 데이터가 서로 뭉쳐 있는 경우에 K-평균 의 적절한 군집개수가 설정됐다고 판단



- ➔ 군집 개수 4개
- ➔ 전체 데이터 평균 실루엣 계수 : 0.65
- ➔ 개별 군집의 평균 실루엣 계수 값이 비교적 균일
- ➔ 군집이 2개인 경우보다는 평균 실루엣 계수 값이 작지만 4개인 경우가 가장 이상적인 군집화 개수로 판단

```
from sklearn.datasets import load_iris
iris=load_iris()
visualize_silhouette([ 2, 3, 4, 5 ], iris.data)
```



#붓꽃 데이터를 이용해 K-평균 수행 시 최적의 군집 개수

군집 개수가 2개일 때, 실루엣 계수가 비교적 균일  
➔ 군집 개수가 3,4,5인 경우 평균 실루엣 계수도 작고 군집 간의 편차가 큼

# #7-2. 군집 평가

## # 이너셔 (Inertia), 엘보우 (Elbow)

**이너셔 (Inertia)**: K-mean 클러스터링의 성능 지표

- 각 샘플과 가장 가까운 센트로이드 사이의 평균 제곱 거리

The diagram shows the formula for Inertia (J) with several annotations: 'number of clusters' points to 'k', 'number of cases' points to 'n', 'case i' points to 'x\_i^{(j)}', 'centroid for cluster j' points to 'c\_j', and 'Distance function' points to the norm symbol. An arrow labeled 'objective function' points to 'J'.

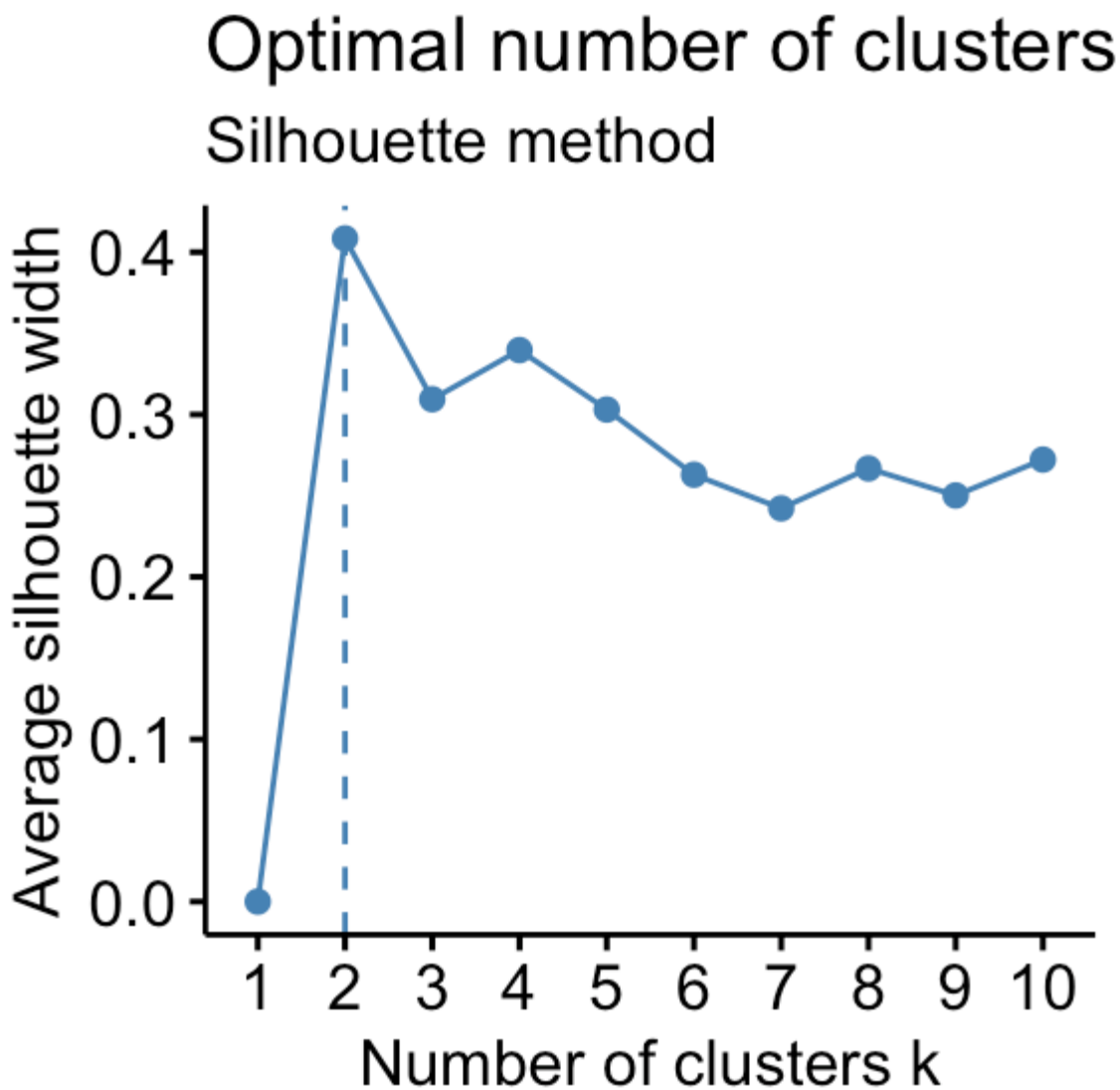
$$J = \sum_{j=1}^k \sum_{i=1}^n \|x_i^{(j)} - c_j\|^2$$

클러스터와 이너셔는 반비례 관계에 있기 때문에, 좋은 모델을 만들기 위해서 이너셔를 성능 지표로 선택하는 것은 좋지 않음

이너셔를 줄이기 위해서 클러스터의 수를 실제 훈련 세트에 있는 클러스터 수보다 많이 생성 -> 클러스터 수가 늘어감에 따라 이너셔가 급격하게 감소하고 어느 지점에서는 완만하게 감소

=> 해당 지점 : 엘보우 (Elbow)

- **엘보우 기법 (Elbow Method)**: 클러스터 개수를 증가시키면서 이너셔의 변화를 관찰하여 최적의 클러스터 개수를 찾는 방법
- 실루엣 점수를 통해 적정 클러스터 수를 결정



# 03 MeanShift





# 03 평균 이동(Mean Shift)

## 평균 이동 군집화

- 중심점을 이동시키며 군집 찾음
- 확률 밀도 함수가 가장 높은 방향으로 중심 이동
- 밀도 피크를 군집의 중심점으로 정함



- ⑤ 개별 데이터들이 모인 중심점을 군집 중심점으로 설정
- ④ 지정된 반복(Iteration) 횟수만큼 전체 데이터에 대해서 KDE 기반으로 데이터를 이동시키면서 군집화 수행
- ③ 모든 데이터를 1~2까지 수행하면서 데이터를 이동. 개별 데이터들이 군집중심점으로 모임.
- ② KDE로 계산된 데이터 분포도가 높은 방향으로 데이터 이동
- ① 개별 데이터의 특정 반경 내에 주변 데이터를 포함한 데이터 분포도를 KDE 기반의 Mean Shift 알고리즘으로 계산

# 03 평균 이동(Mean Shift)

## 커널 밀도 추정(KDE)

: 데이터의 확률 밀도 함수 추정 방법

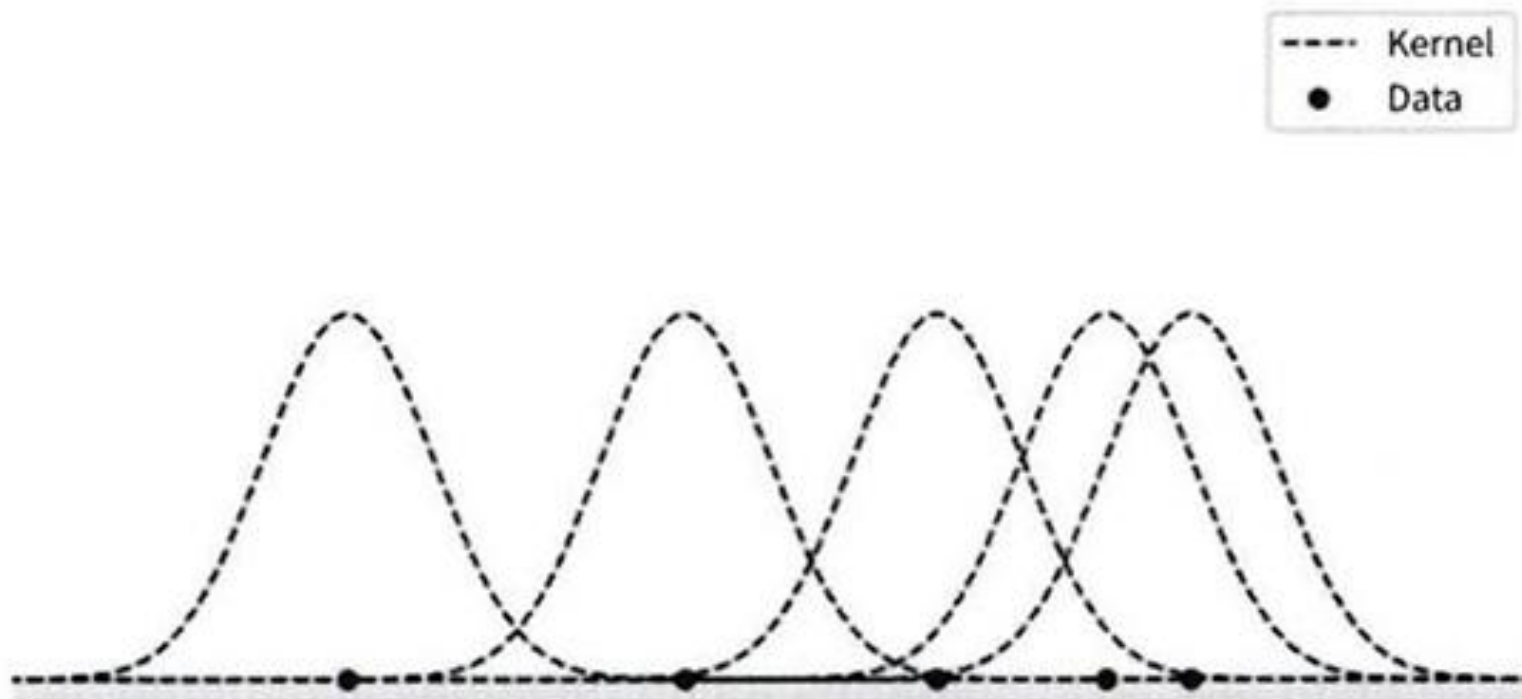
각 데이터 포인트마다 커널 함수 적용,

→ 모두 더한 뒤 데이터 개수로 나눠 확률 밀도 함수 만들

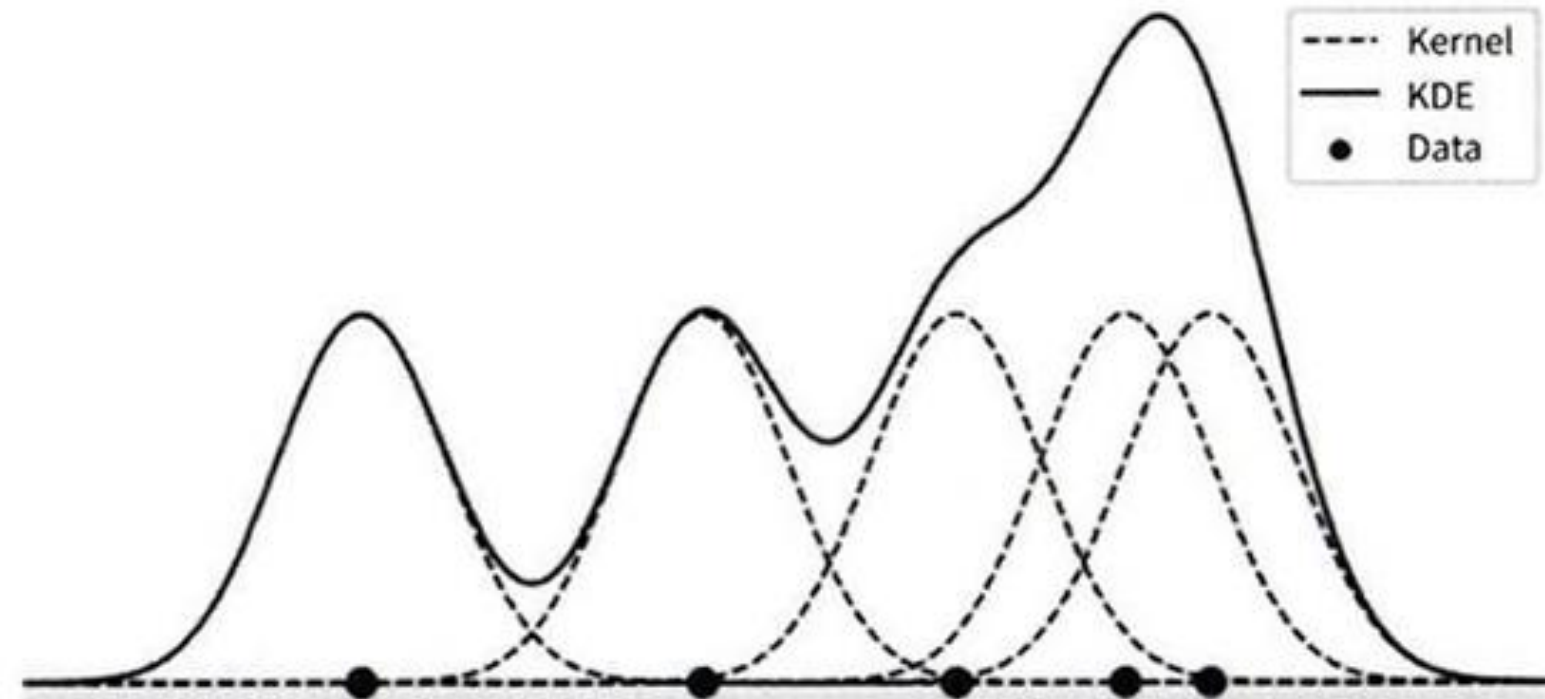
$$\text{KDE} = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

여기서  $K$ 는 커널 함수,  $x$ 는 확률 변수,  $x_i$ 는 관측값,  $h$ 는 대역폭(bandwidth)입니다

개별 관측 데이터에 가우시안 커널 함수 적용



가우시안 커널 함수 적용 후 합산





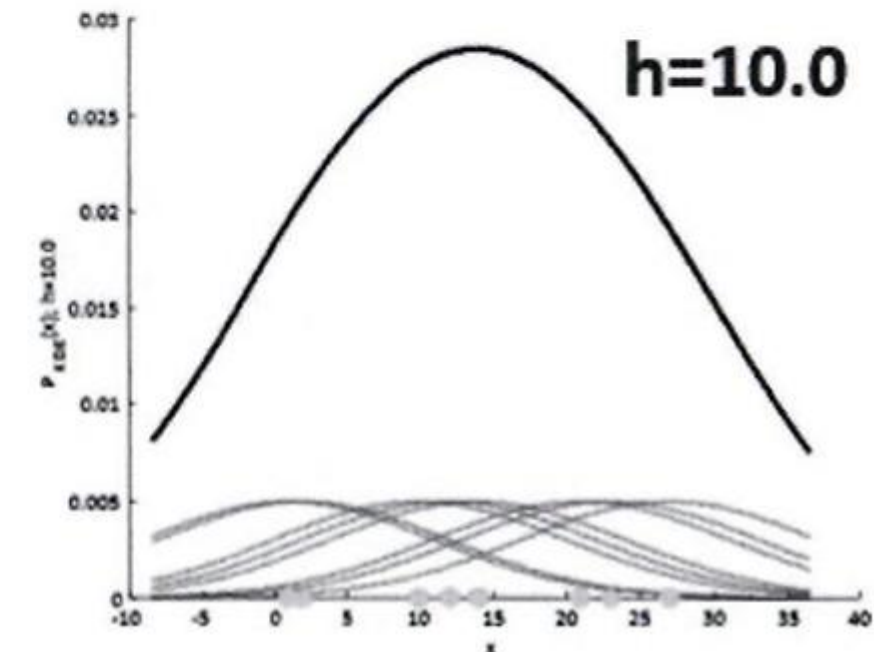
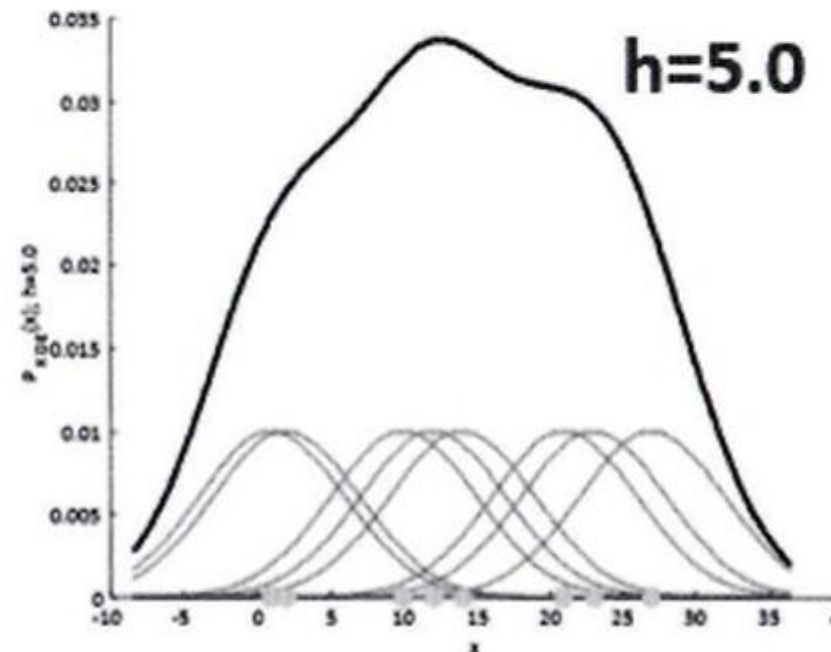
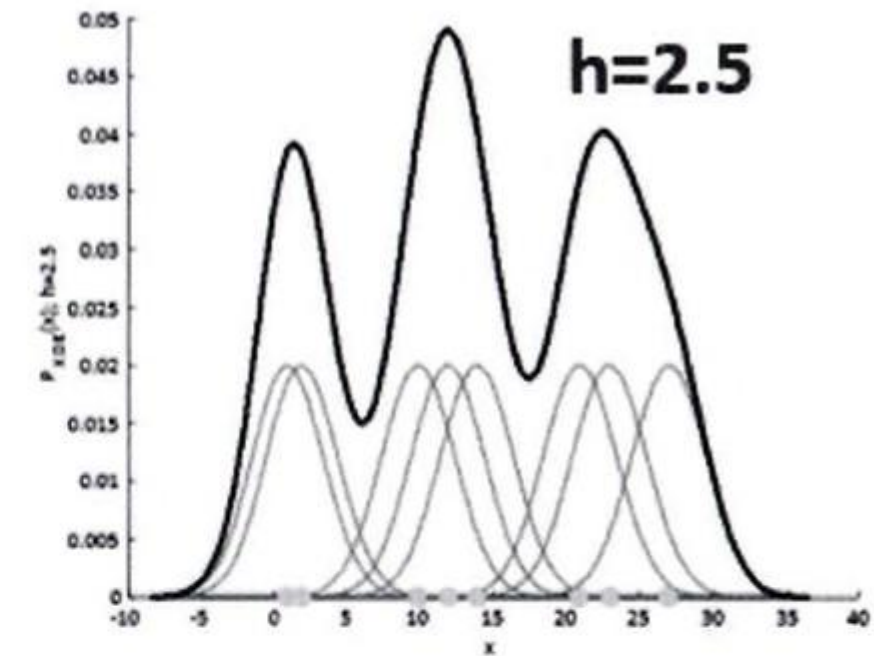
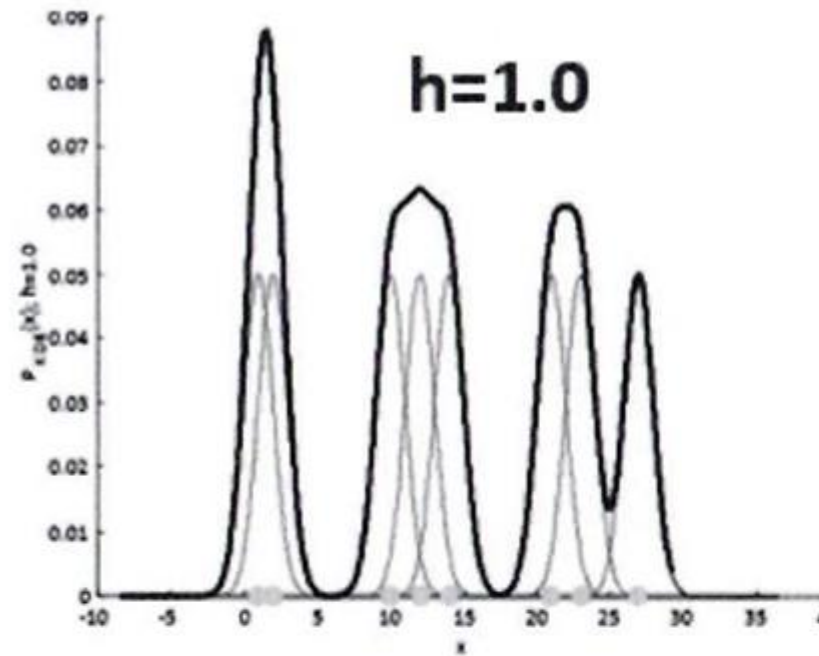
# 03 평균 이동(Mean Shift)

## 대역폭 (Bandwidth)

: 곡선의 부드러움(평활화 정도)을 결정

- $h$  작음  $\rightarrow$  KDE 뾰족해짐
  - 과적합 가능
- $h$  큼  $\rightarrow$  KDE 완만해짐
  - 과소적합 가능

$\Rightarrow$  적절한 대역폭 선택!



# 03 평균 이동(Mean Shift)

## Bandwith 값에 따른 군집 개수 변화

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import MeanShift

X, y = make_blobs(n_samples=200, n_features=2, centers=3, cluster_std=0.7, random_state=0)

meanshift = MeanShift(bandwidth = 0.8)
cluster_labels = meanshift.fit_predict(X)
print('cluster lables 유형:', np.unique(cluster_labels))

cluster lables 유형: [0 1 2 3 4 5]
```

- make\_blobs(), 3개의 군집 데이터
- Bandwidth = 0.8
- 6개의 군집

- Bandwidth = 1.0
  - 3개의 군집

```
meanshift = MeanShift(bandwidth=1)
cluster_labels = meanshift.fit_predict(X)
print('cluster lables 유형:', np.unique(cluster_labels))

cluster lables 유형: [0 1 2]
```

# 03 평균 이동(Mean Shift)

## 최적의 Bandwidth 값 찾기

```
from sklearn.cluster import estimate_bandwidth

bandwidth = estimate_bandwidth(X)
print('bandwidth 값:', round(bandwidth, 3))
```

```
bandwidth 값: 1.816
```

```
import pandas as pd

clusterDF = pd.DataFrame(data=X, columns=['ftr1', 'ftr2'])
clusterDF['target'] = y

best_bandwidth = estimate_bandwidth(X)

meanshift = MeanShift(bandwidth=best_bandwidth)
cluster_labels = meanshift.fit_predict(X)
print('cluster labels 유형:', np.unique(cluster_labels))
```

```
cluster labels 유형: [0 1 2]
```

sklearn – estimate\_bandwidth 함수 제공

- feature 데이터 입력
- 데이터 분포 고려, 적절한 bandwidth 값 반환

- Bandwidth = 1.816
- MeanShift의 bandwidth 파라미터로 입력
- 데이터에 가장 적합한 군집 개수

# 03 평균 이동(Mean Shift)

## 군집 중심 좌표

```
import matplotlib.pyplot as plt
%matplotlib inline

clusterDF['meanshift_label'] = cluster_labels
centers = meanshift.cluster_centers_
unique_labels = np.unique(cluster_labels)
markers = ['o', 's', '^', 'x', '*']

for label in unique_labels:
    label_cluster = clusterDF[clusterDF['meanshift_label'] == label]
    center_x_y = centers[label]

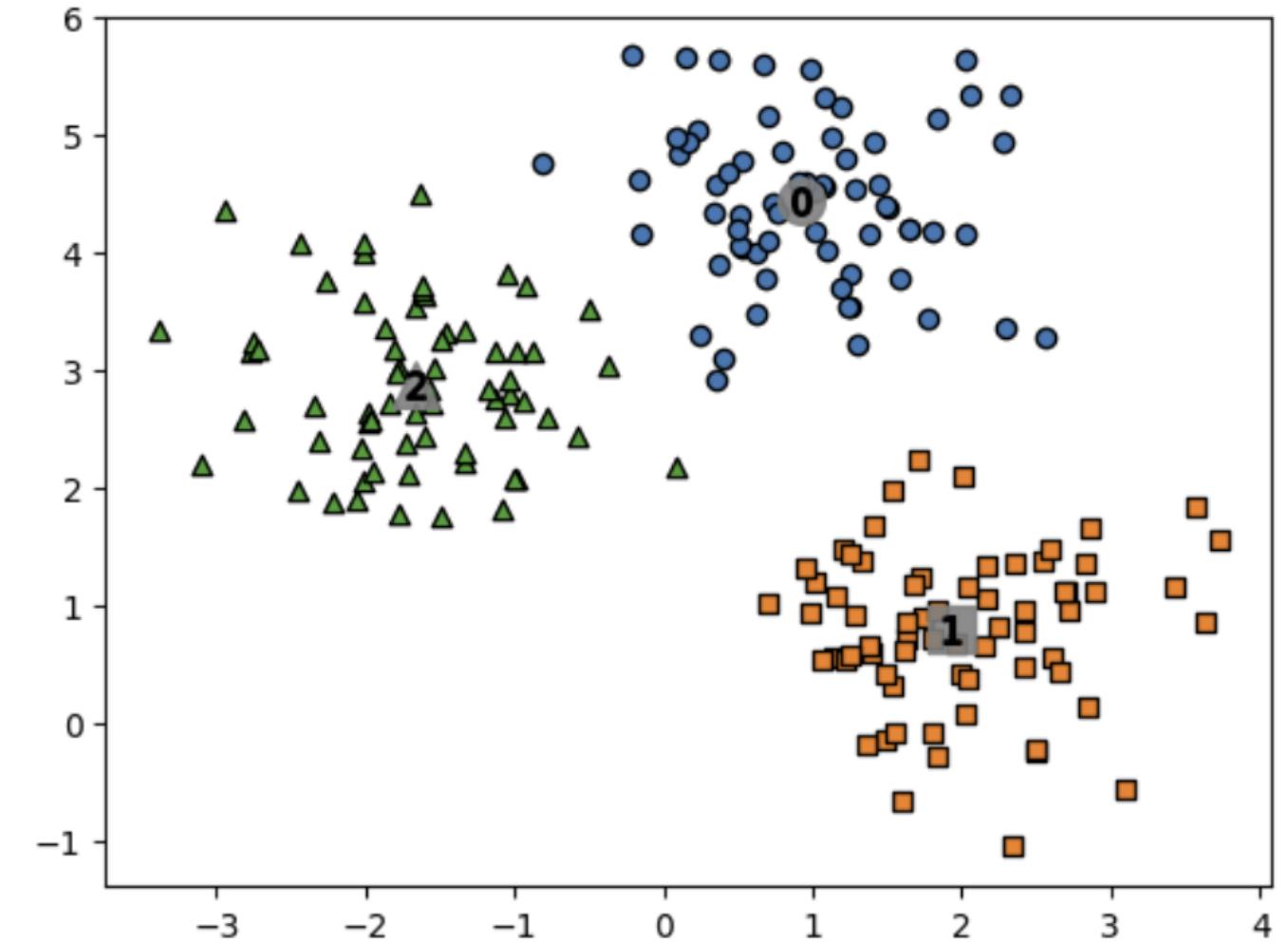
    plt.scatter(x=label_cluster['ftr1'], y=label_cluster['ftr2'], edgecolor='k', marker=markers[label])
    plt.scatter(x=center_x_y[0], y=center_x_y[1], s=200, color='gray', alpha=0.9, marker=markers[label])
    plt.scatter(x=center_x_y[0], y=center_x_y[1], s=70, color='k', edgecolor='k', marker='$_d$' % label)

plt.show()
```

```
print(clusterDF.groupby('target')['meanshift_label'].value_counts())
```

| target | meanshift_label |    |
|--------|-----------------|----|
| 0      | 0               | 67 |
| 1      | 1               | 67 |
| 2      | 2               | 66 |

Name: count, dtype: int64



# 04 GMM



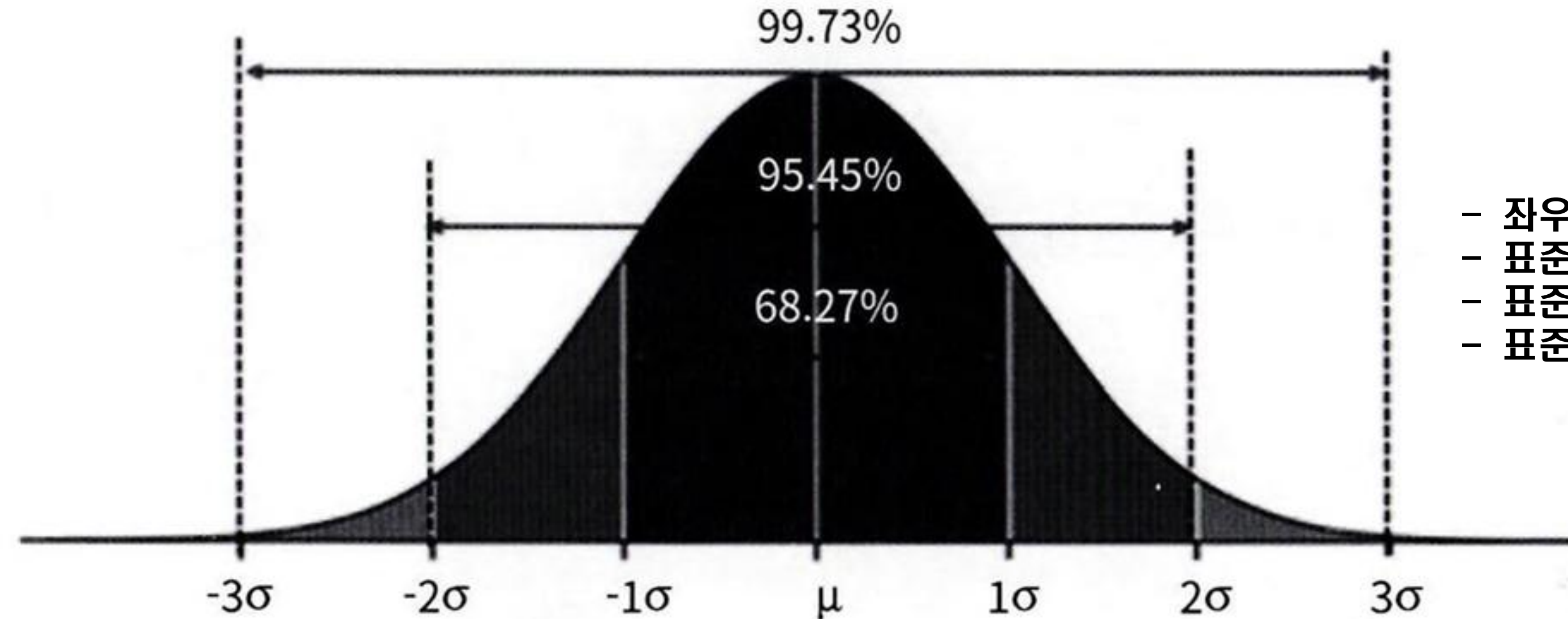


# 04 GMM(Gaussian Mixture Model)

## GMM 군집화

: 데이터가 여러 가우시안 분포로 만들어졌다고 가정

- 그 안의 각각의 정규 분포 찾아냄



- 좌우 대칭 종모양
- 표준편차 1 이내 68.27%
- 표준편차 2 이내 95.45%
- 표준편차 3 이내 99.73%



# 04 GMM(Gaussian Mixture Model)



모수 추정

1. 각 정규 분포의 평균과 분산
2. 각 데이터가 어떤 정규 분포에 속할 확률

→ EM (Expectation and Maximization)

→ sklearn : GaussianMixture

# 04 GMM(Gaussian Mixture Model)

```
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components = 3, random_state=0).fit(iris.data)
gmm_cluster_labels = gmm.predict(iris.data)
```

```
irisDF['gmm_cluster'] = gmm_cluster_labels
irisDF['target'] = iris.target
```

```
iris_result = irisDF.groupby(['target'])['gmm_cluster'].value_counts()
print(iris_result)
```

```
target  gmm_cluster
0       1           50
1       0           45
        2            5
2       2           50
Name: count, dtype: int64
```

## GMM 군집화

**n\_components = 3**  
: 군집의 개수

**fit, predict → 군집 결정**  
**결과 gmm\_cluster에 저장**

**Target 0 → cluster 0**  
**Target 2 → cluster 1**

**Target1 → cluster 1: 5개**  
**cluster 2: 45개**

```
kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=300, random_state=0).fit(iris.data)
kmeans_cluster_labels = kmeans.predict(iris.data)
irisDF['kmeans_cluster'] = kmeans_cluster_labels
iris_result = irisDF.groupby(['target'])['kmeans_cluster'].value_counts()
print(iris_result)
```

```
target  kmeans_cluster
0       1             50
1       0             47
        2              3
2       2             36
        0             14
Name: count, dtype: int64
```

## K-평균 군집화

**Target 0 → cluster 1**

**Target1 → cluster 0: 47개**  
**cluster 2: 3개**

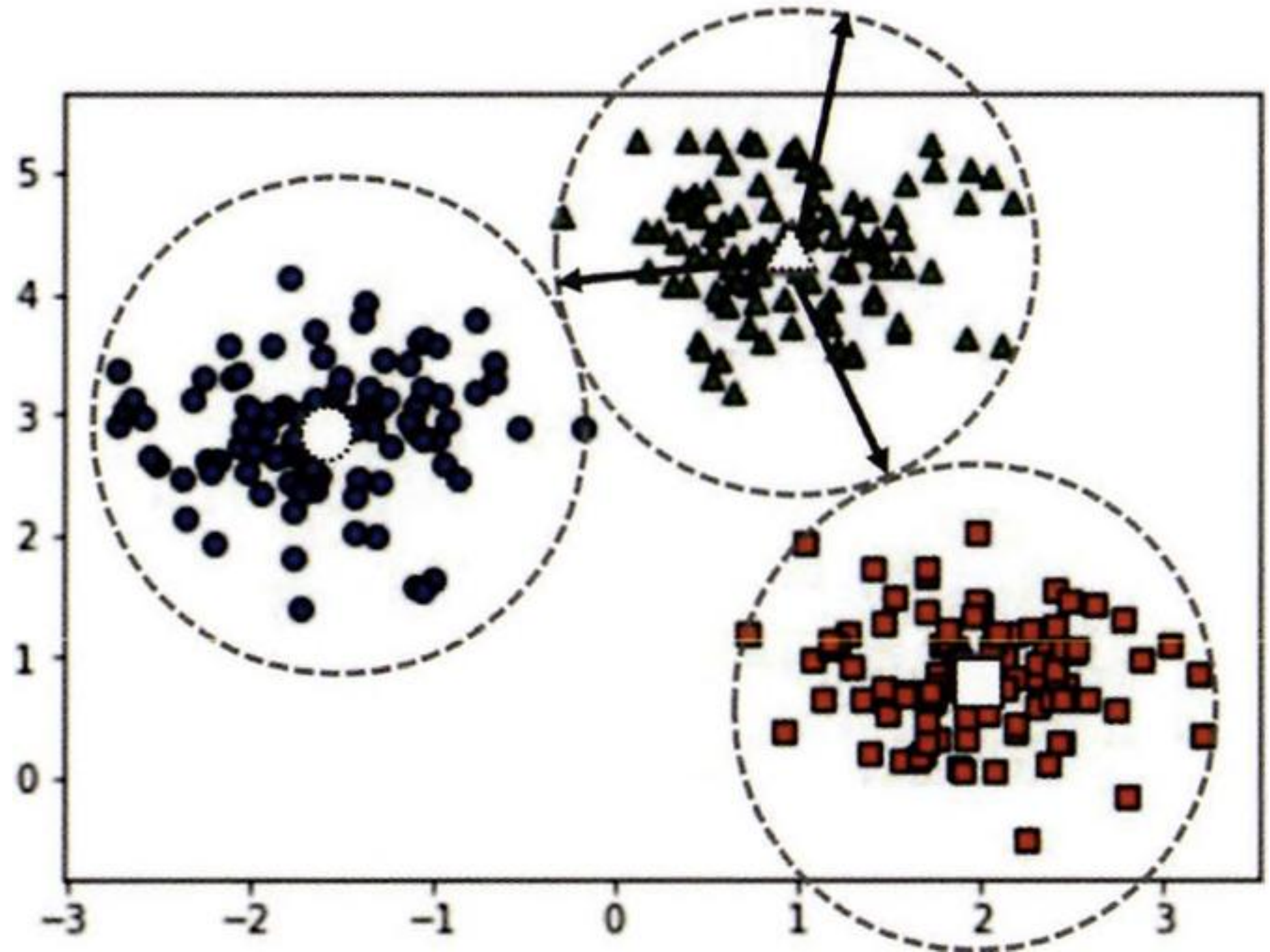
**Target1 → cluster 0: 14개**  
**cluster 2: 36개**

# 04 GMM(Gaussian Mixture Model)

## K-평균 군집화

: 데이터 세트 원형 범위에수록 효과적

- make\_blobs()로 군집 수 3개
- cluster\_std = 0.5



# 04 GMM(Gaussian Mixture Model)

```
def visualize_cluster_plot(clusterobj, dataframe, label_name, iscenter=True):
    if iscenter:
        centers = clusterobj.cluster_centers_

    unique_labels = np.unique(dataframe[label_name].values)
    markers=['o', 's', '^', 'x', '*']
    isNoise=False

    for label in unique_labels:
        label_cluster = dataframe[dataframe[label_name]==label]
        if label == -1:
            cluster_legend = 'Noise'
            isNoise=True
        else:
            cluster_legend = 'Cluster '+str(label)

        plt.scatter(x=label_cluster['ftr1'], y=label_cluster['ftr2'], s=70, #
                    edgecolor='k', marker=markers[label], label=cluster_legend)

        if iscenter:
            center_x_y = centers[label]
            plt.scatter(x=center_x_y[0], y=center_x_y[1], s=250, color='white',
                        alpha=0.9, edgecolor='k', marker=markers[label])
            plt.scatter(x=center_x_y[0], y=center_x_y[1], s=70, color='k', #
                        edgecolor='k', marker='$%d$' % label)

    if isNoise:
        legend_loc='upper center'
    else: legend_loc='upper right'

    plt.legend(loc=legend_loc)
    plt.show()
```

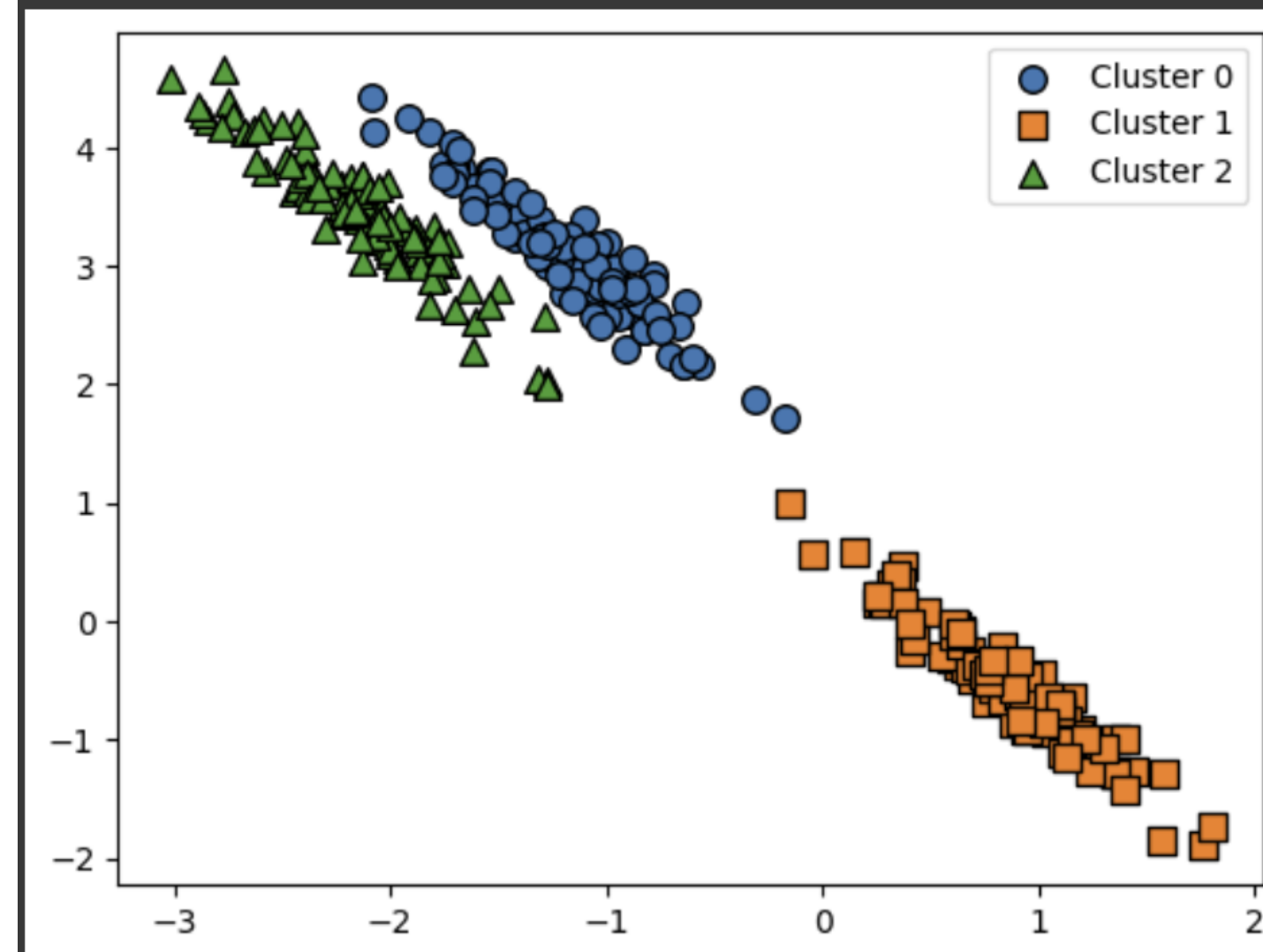
```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=300, n_features=2, centers=3, cluster_std=0.5, random_state=0)

transformation = [[0.60834549, -0.63667341], [-0.40887718, 0.85253229]]

X_aniso = np.dot(X, transformation)

clusterDF = pd.DataFrame(data=X_aniso, columns=['ftr1', 'ftr2'])
clusterDF['target'] = y

visualize_cluster_plot(None, clusterDF, 'target', iscenter=False)
```



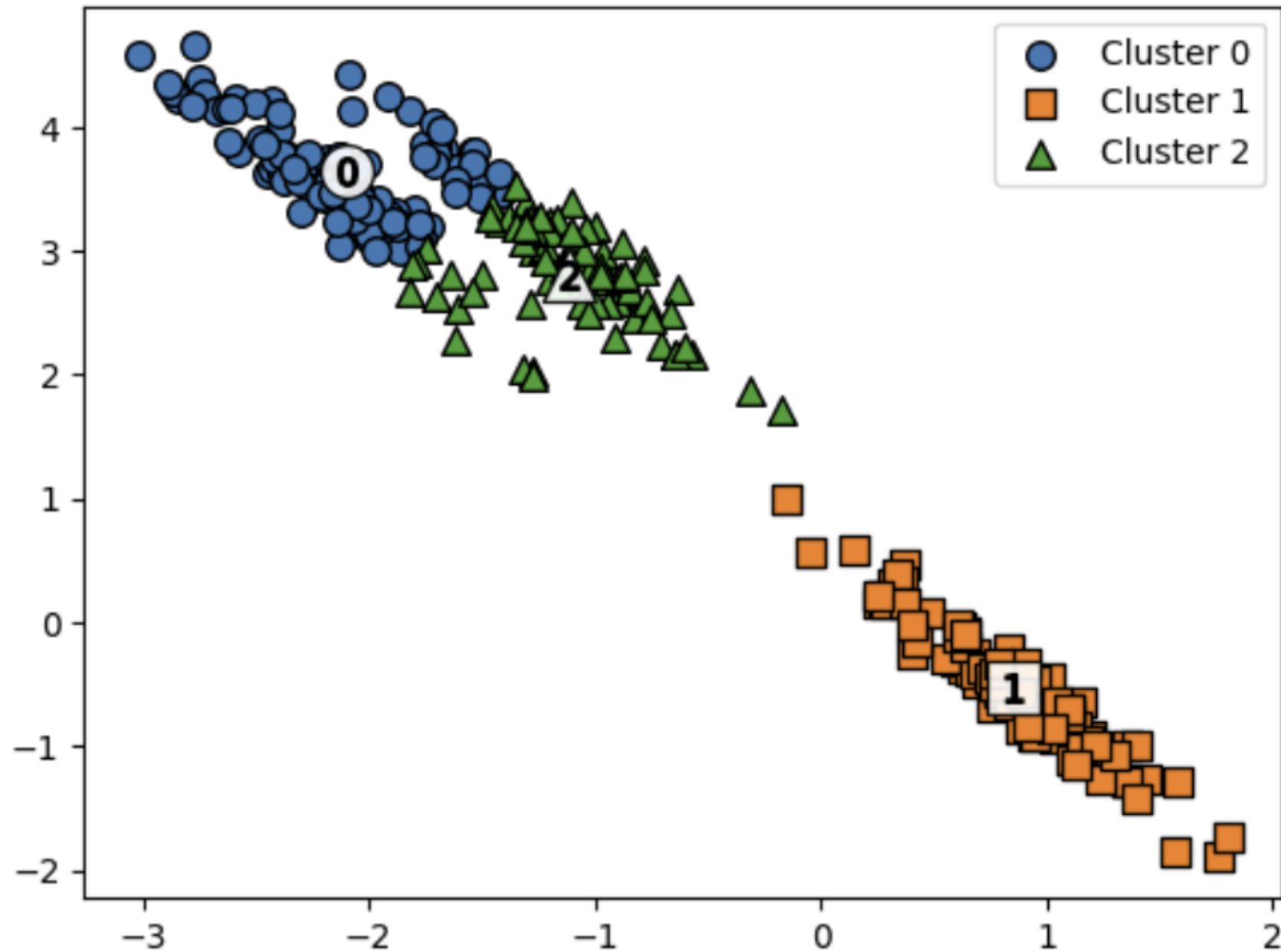


# 04 GMM(Gaussian Mixture Model)

## K-평균 군집화

```
kmeans = KMeans(3, random_state=0)
kmeans_label = kmeans.fit_predict(X_aniso)
clusterDF['kmeans_label'] = kmeans_label

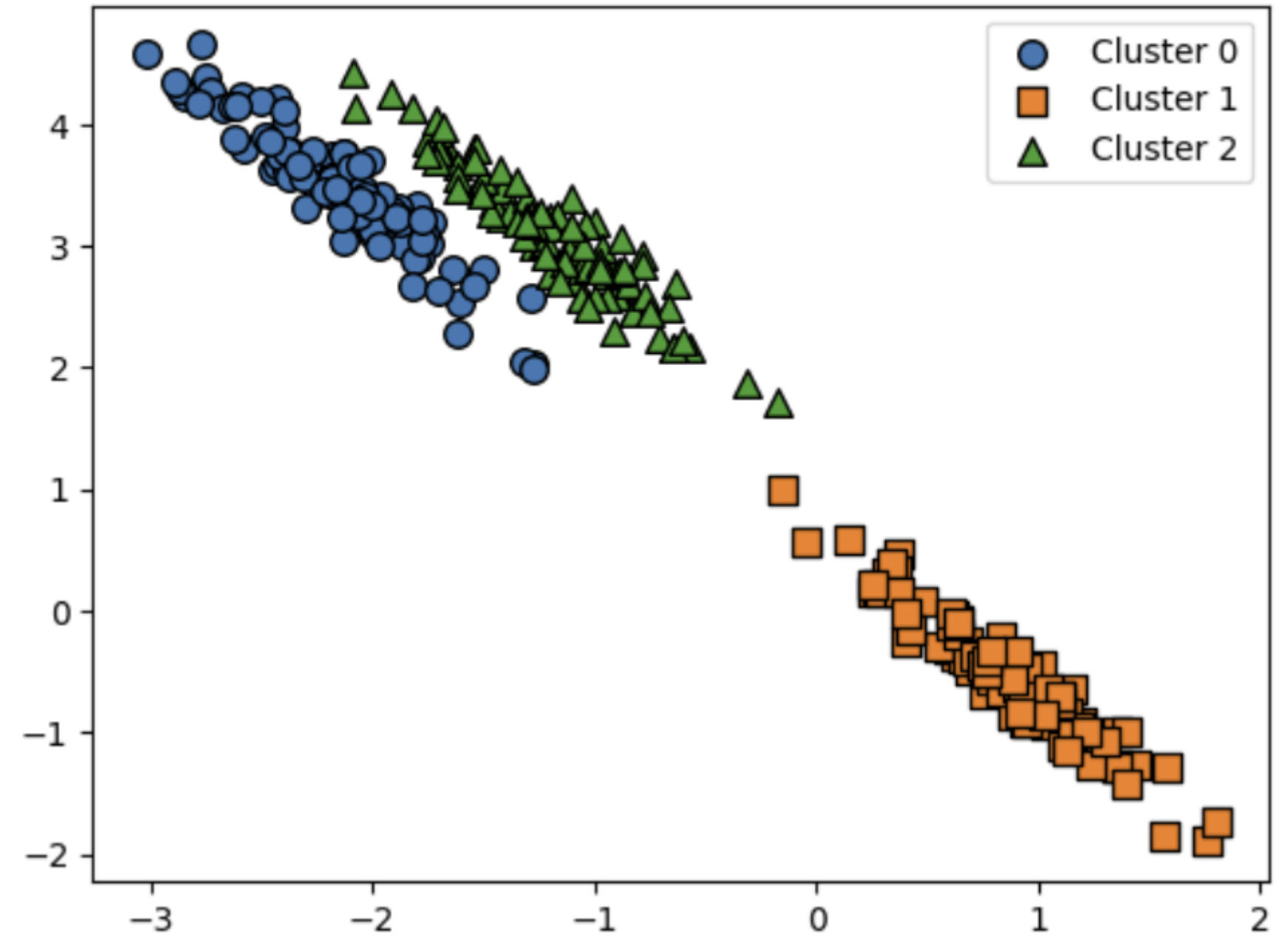
visualize_cluster_plot(kmeans, clusterDF, 'kmeans_label', iscenter=True)
```



## GMM 군집화

```
gmm = GaussianMixture(n_components=3, random_state=0)
gmm_label = gmm.fit(X_aniso).predict(X_aniso)
clusterDF['gmm_label'] = gmm_label

visualize_cluster_plot(gmm, clusterDF, 'gmm_label', iscenter=False)
```



# 04 GMM(Gaussian Mixture Model)

## Kmeans, GMM의 군집 Label값 비교

```
print('### KMeans Clustering ###')
print(clusterDF.groupby('target')['kmeans_label'].value_counts())
print('\n### Gaussian Mixture Clustering ###')
print(clusterDF.groupby('target')['gmm_label'].value_counts())
```

```
### KMeans Clustering ###
target  kmeans_label
0        2           73
         0           27
1        1          100
2        0           86
         2           14
Name: count, dtype: int64
```

```
### Gaussian Mixture Clustering ###
target  gmm_label
0        2          100
1        1          100
2        0          100
Name: count, dtype: int64
```

**Kmeans – 군집1만 정확히 매핑**

**GMM – 모든 군집이 target값과 잘 매핑**

**: Kmeans보다 유연하게 적용**

**: 군집화 수행 시간 짧**



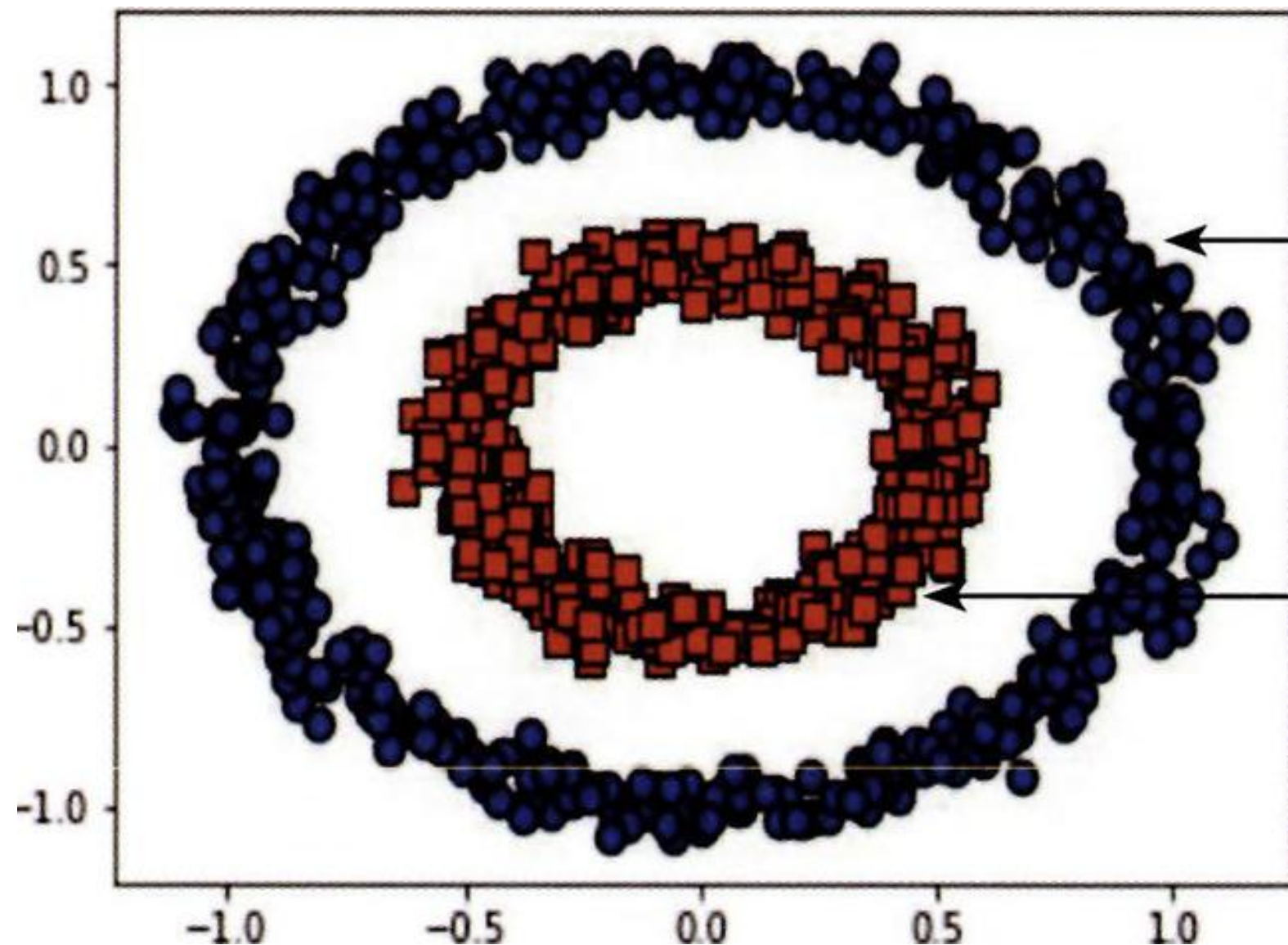
# 05 DBSCAN



# 05 DBSCAN

## 파라미터

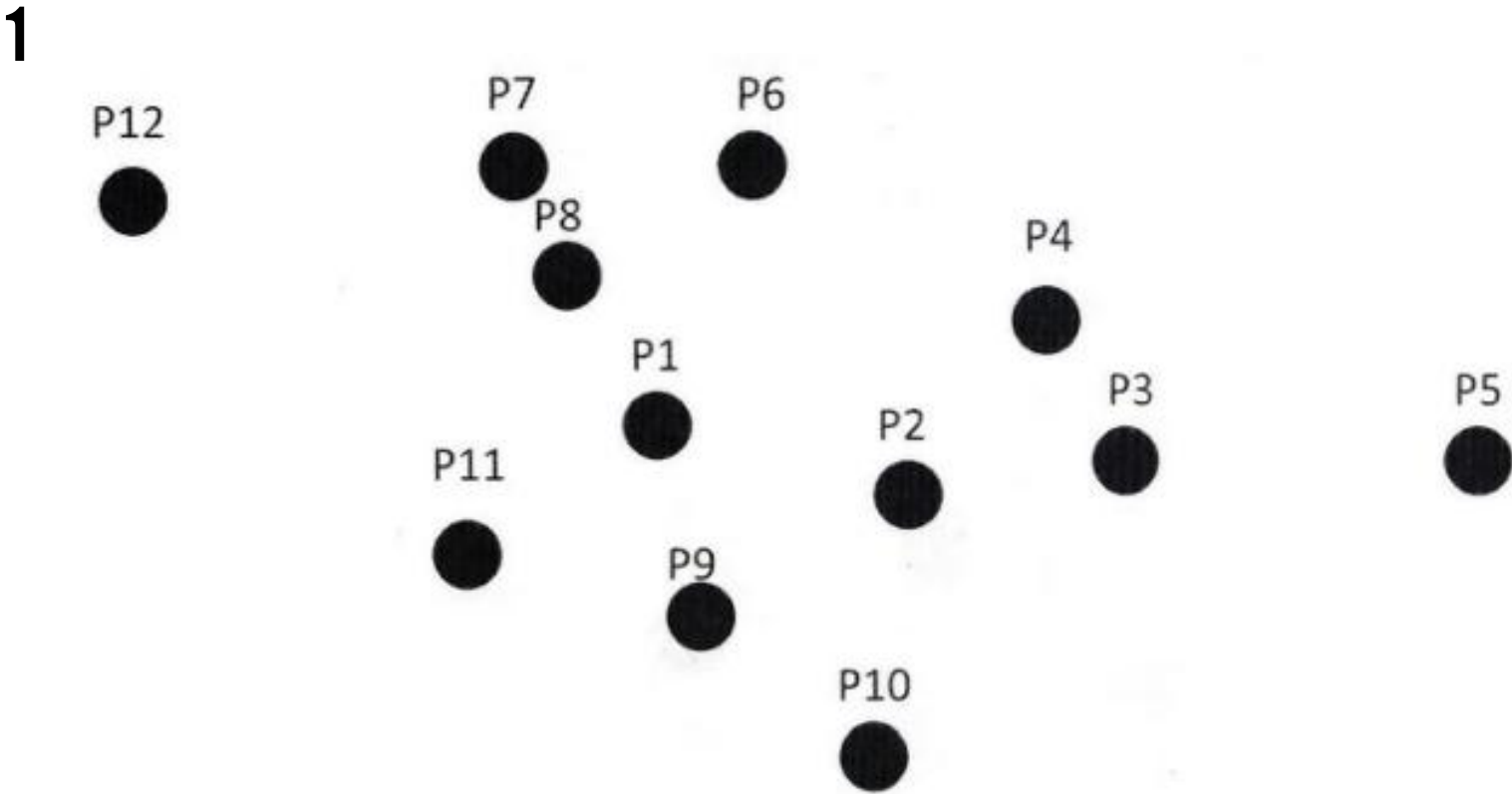
- **입실론 주변 영역(epsilon)**  
: 개별 데이터 중심 입실론 반경의 원형 영역
- **최소 데이터 개수(min points)**  
: 개별 데이터 입실론 주변 영역에 포함되는 타 데이터 개수



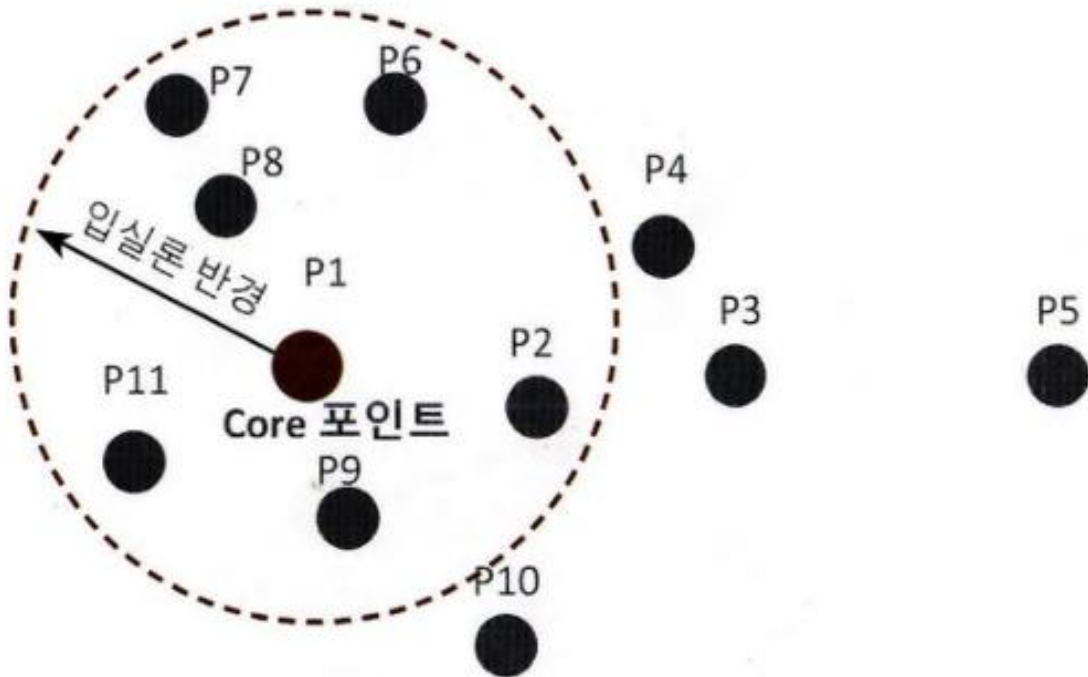
## 데이터 포인트

- **핵심 포인트(core point)**  
: 주변 영역 내 최소 데이터 개수 이상의 타 데이터 있는 경우
- **이웃 포인트(neighbor point)**  
: 주변 영역 내 위치한 타 데이터
- **경계 포인트(border point)**  
: 주변 영역 내 최소 데이터 개수 미만  
: 핵심 포인트를 이웃 포인트로 가지는 경우
- **잡음 포인트(noise point)**  
: 최소 데이터 미만의 이웃 포인트  
: 핵심 포인트도 이웃 포인트도 가지지 않는 경우

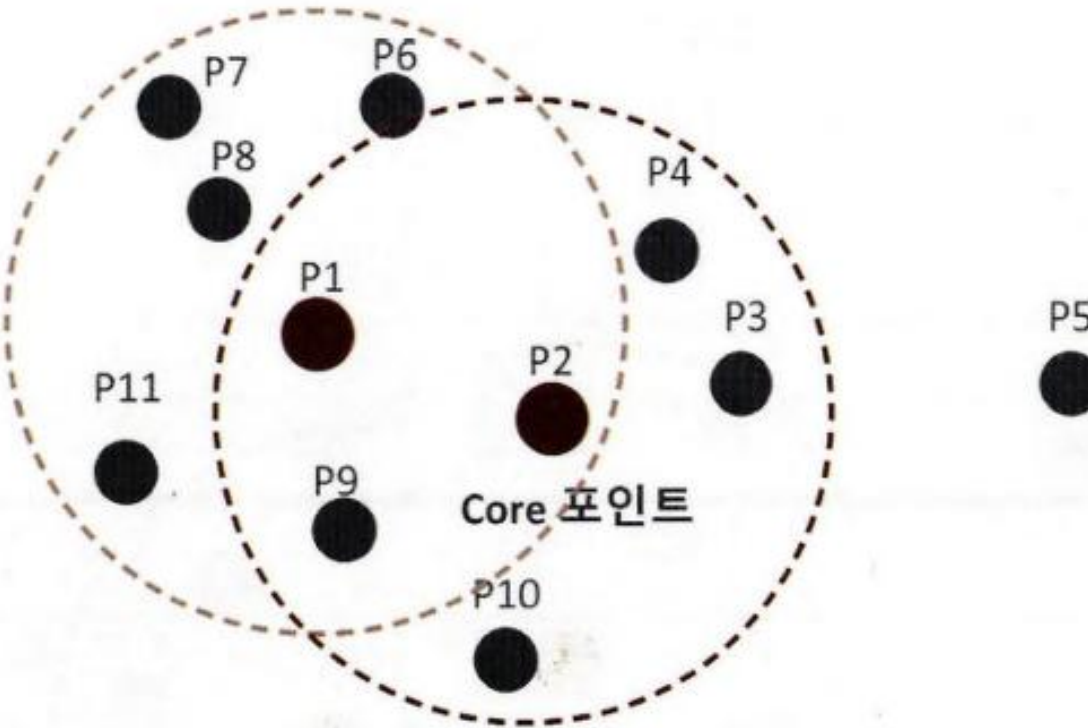
# 05 DBSCAN



2



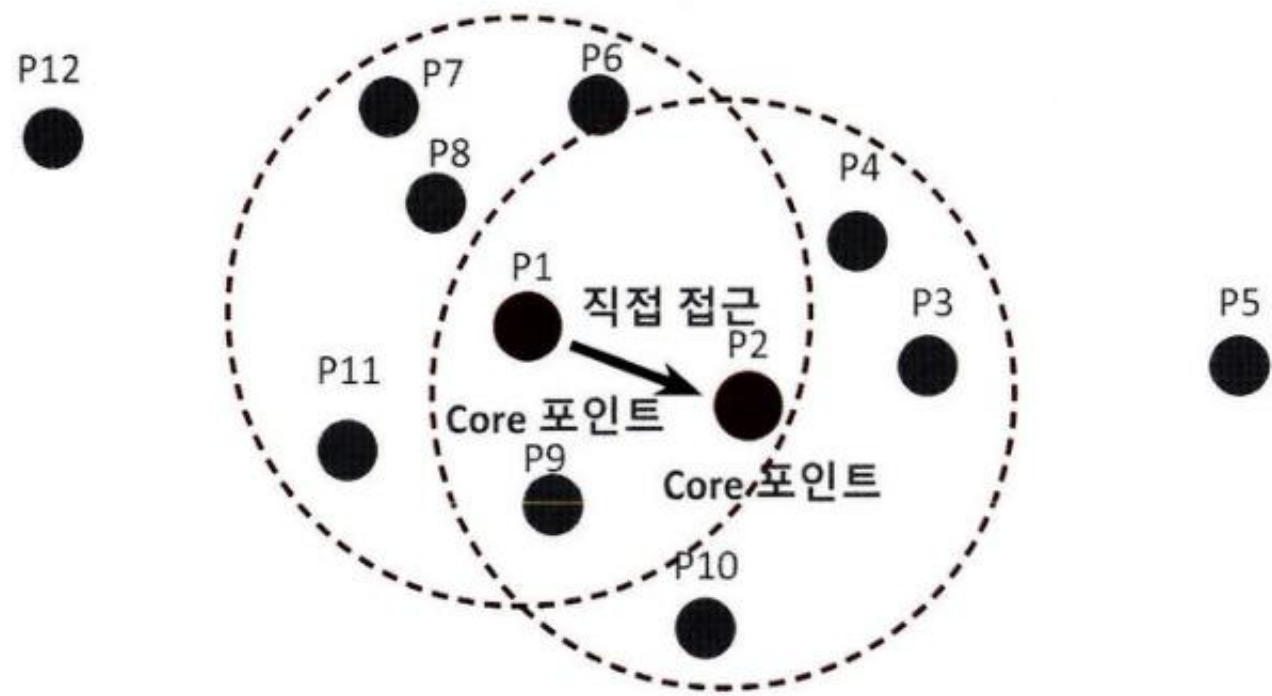
3



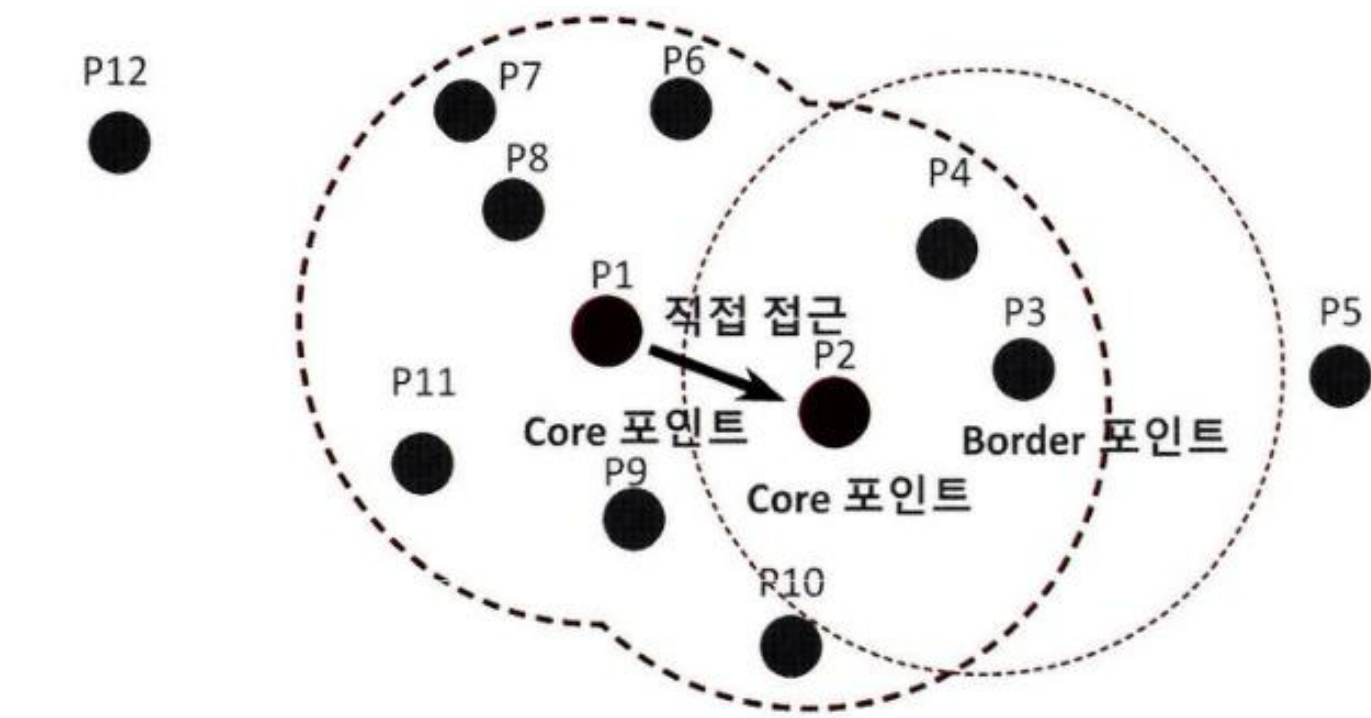


# 05 DBSCAN

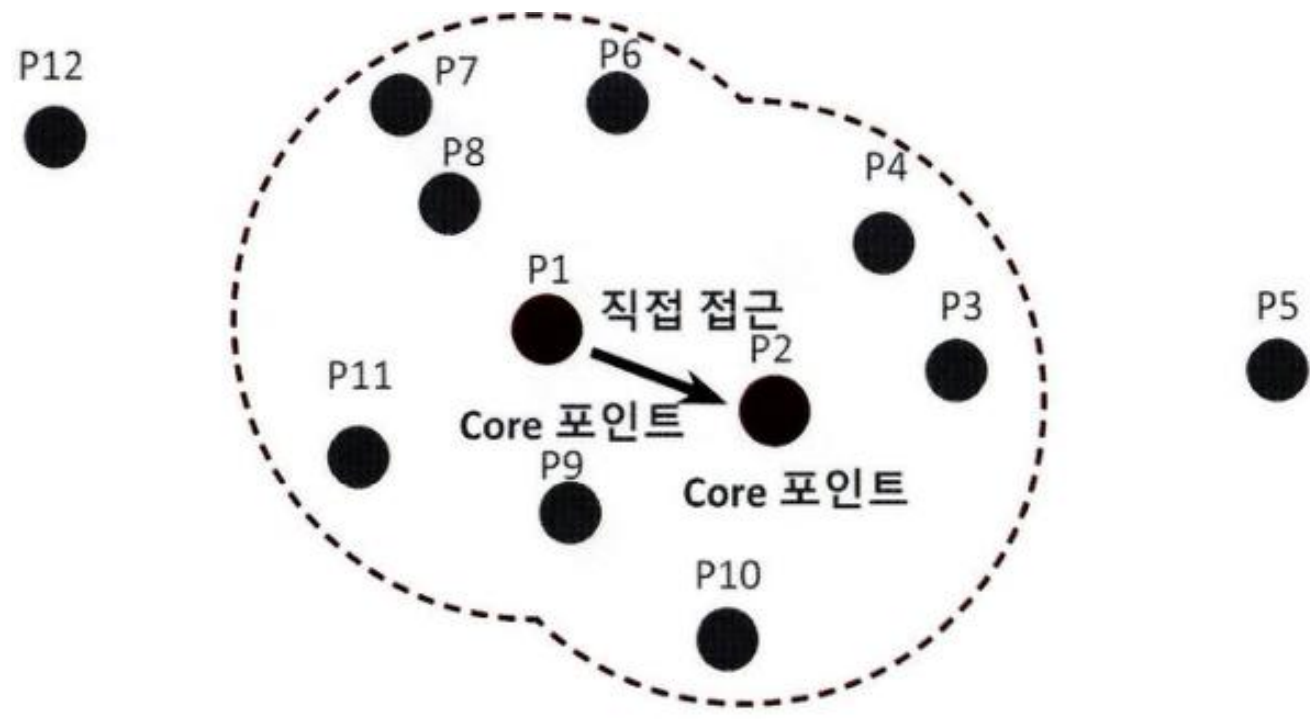
4



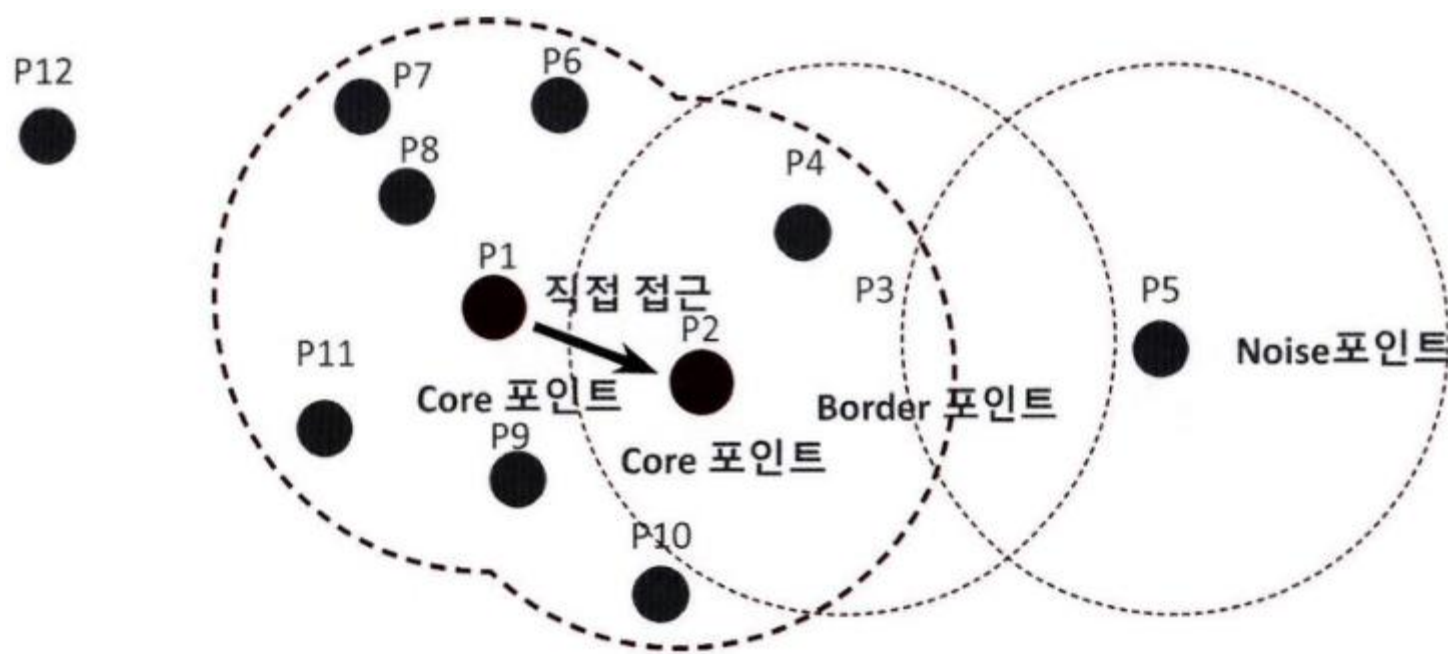
6



5



7



# 05 DBSCAN

## DBSCAN 군집화

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.6, min_samples=8, metric='euclidean')
dbscan_labels = dbscan.fit_predict(iris.data)

irisDF['dbscan_cluster'] = dbscan_labels
irisDF['target'] = iris.target

iris_result = irisDF.groupby(['target'])['dbscan_cluster'].value_counts()
print(iris_result)
```

| target | dbscan_cluster |    |
|--------|----------------|----|
| 0      | 0              | 49 |
|        | -1             | 1  |
| 1      | 1              | 46 |
|        | -1             | 4  |
| 2      | 1              | 42 |
|        | -1             | 8  |

Name: count, dtype: int64

-1: 노이즈

- 0과 1 두 개의 군집으로 군집화
- Target 값 3가지, 군집 2개  
: 군집화 효율 떨어지는 것은 아님

# 05 DBSCAN

```
### 클러스터 결과를 담은 DataFrame과 사이킷런의 Cluster 객체등을 인자로 받아 클러스터링 결과를 시각화하는 함수
def visualize_cluster_plot(clusterobj, dataframe, label_name, iscenter=True):
    if iscenter :
        centers = clusterobj.cluster_centers_

    unique_labels = np.unique(dataframe[label_name].values)
    markers=['o', 's', '^', 'x', '*']
    isNoise=False

    for label in unique_labels:
        label_cluster = dataframe[dataframe[label_name]==label]
        if label == -1:
            cluster_legend = 'Noise'
            isNoise=True
        else :
            cluster_legend = 'Cluster '+str(label)

        plt.scatter(x=label_cluster['ftr1'], y=label_cluster['ftr2'], s=70, #
                    edgecolor='k', marker=markers[label], label=cluster_legend)

        if iscenter:
            center_x_y = centers[label]
            plt.scatter(x=center_x_y[0], y=center_x_y[1], s=250, color='white',
                        alpha=0.9, edgecolor='k', marker=markers[label])
            plt.scatter(x=center_x_y[0], y=center_x_y[1], s=70, color='k', #
                        edgecolor='k', marker='*')

    if isNoise:
        legend_loc='upper center'
    else: legend_loc='upper right'

    plt.legend(loc=legend_loc)
    plt.show()
```

- 군집화 결과 2차원 평면에 시각화 위해 PCA로 압축
- \* : 노이즈 데이터

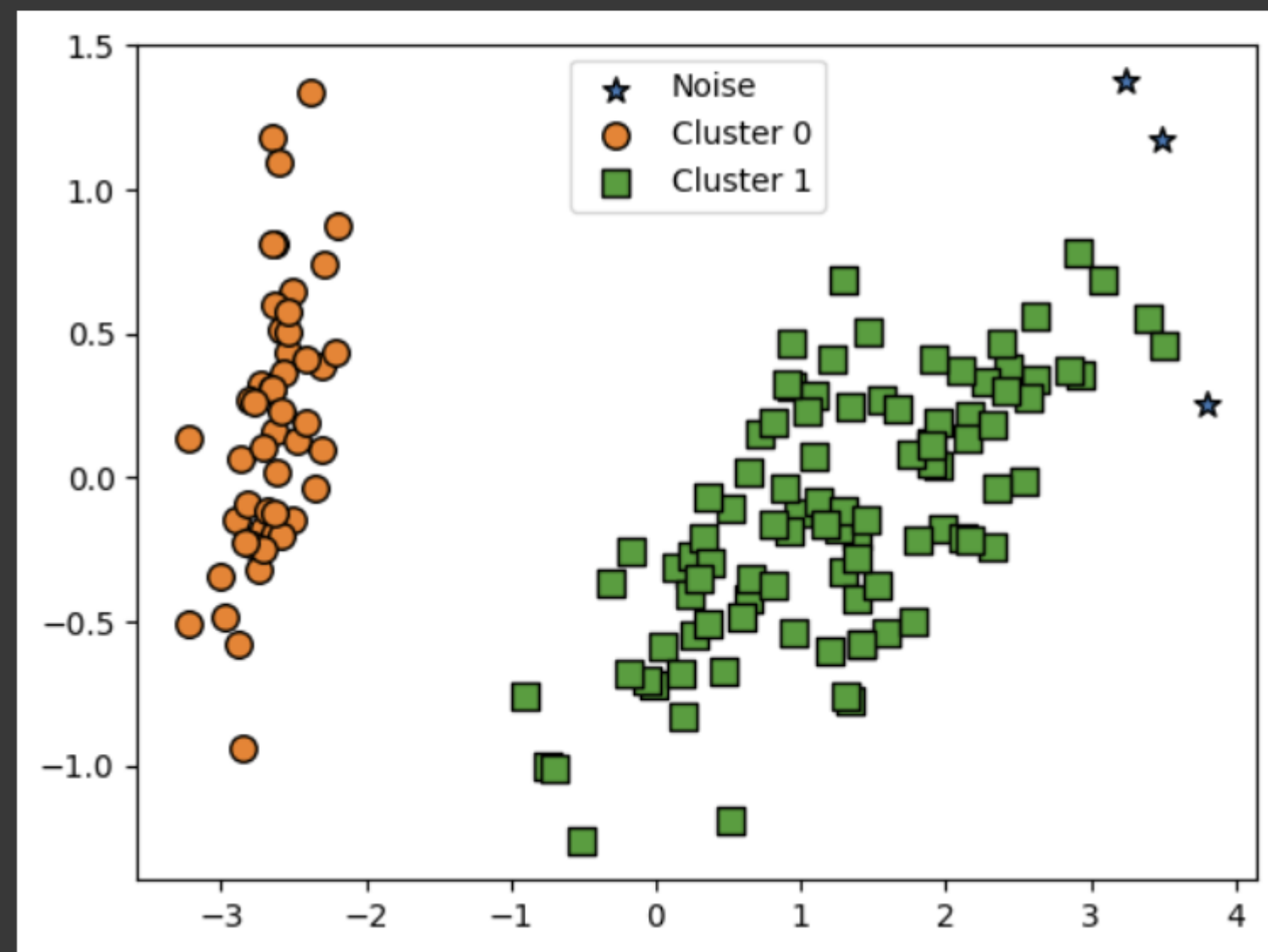
- eps 값 0.6 → 0.8  
: 반경 넓어지며 노이즈 데이터 감소

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=0)
pca_transformed = pca.fit_transform(iris.data)

irisDF['ftr1'] = pca_transformed[:,0]
irisDF['ftr2'] = pca_transformed[:,1]

visualize_cluster_plot(dbscan, irisDF, 'dbscan_cluster', iscenter=False)
```





# 05 DBSCAN

```
dbscan = DBSCAN(eps=0.6, min_samples=16, metric='euclidean')
dbscan_labels = dbscan.fit_predict(iris.data)

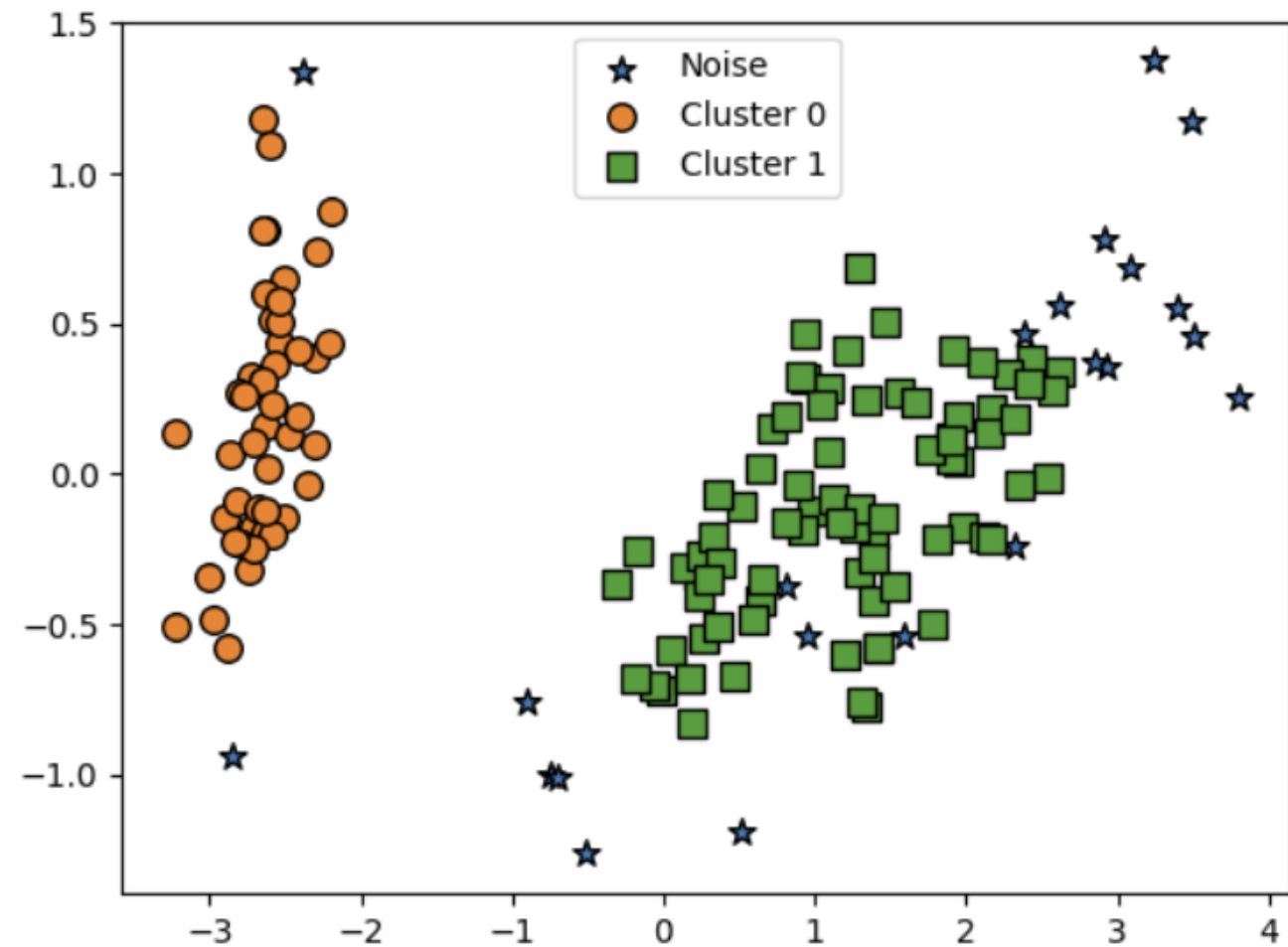
irisDF['dbscan_cluster'] = dbscan_labels
irisDF['target'] = iris.target

iris_result = irisDF.groupby(['target'])['dbscan_cluster'].value_counts()
print(iris_result)

visualize_cluster_plot(dbscan, irisDF, 'dbscan_cluster', iscenter=False)
```

| target | dbscan_cluster |    |
|--------|----------------|----|
| 0      | 0              | 48 |
|        | -1             | 2  |
| 1      | 1              | 44 |
|        | -1             | 6  |
| 2      | 1              | 36 |
|        | -1             | 14 |

Name: count, dtype: int64



- eps = 0.6
- min\_samples = 16
- : min\_samples 커지면 노이즈 데이터 증가

# 05 DBSCAN

`make_circles()`

: 2개의 feature만을 생성

: 별도의 feature 개수 지정 필요X

`noise`

: 노이즈 데이터 세트 비율

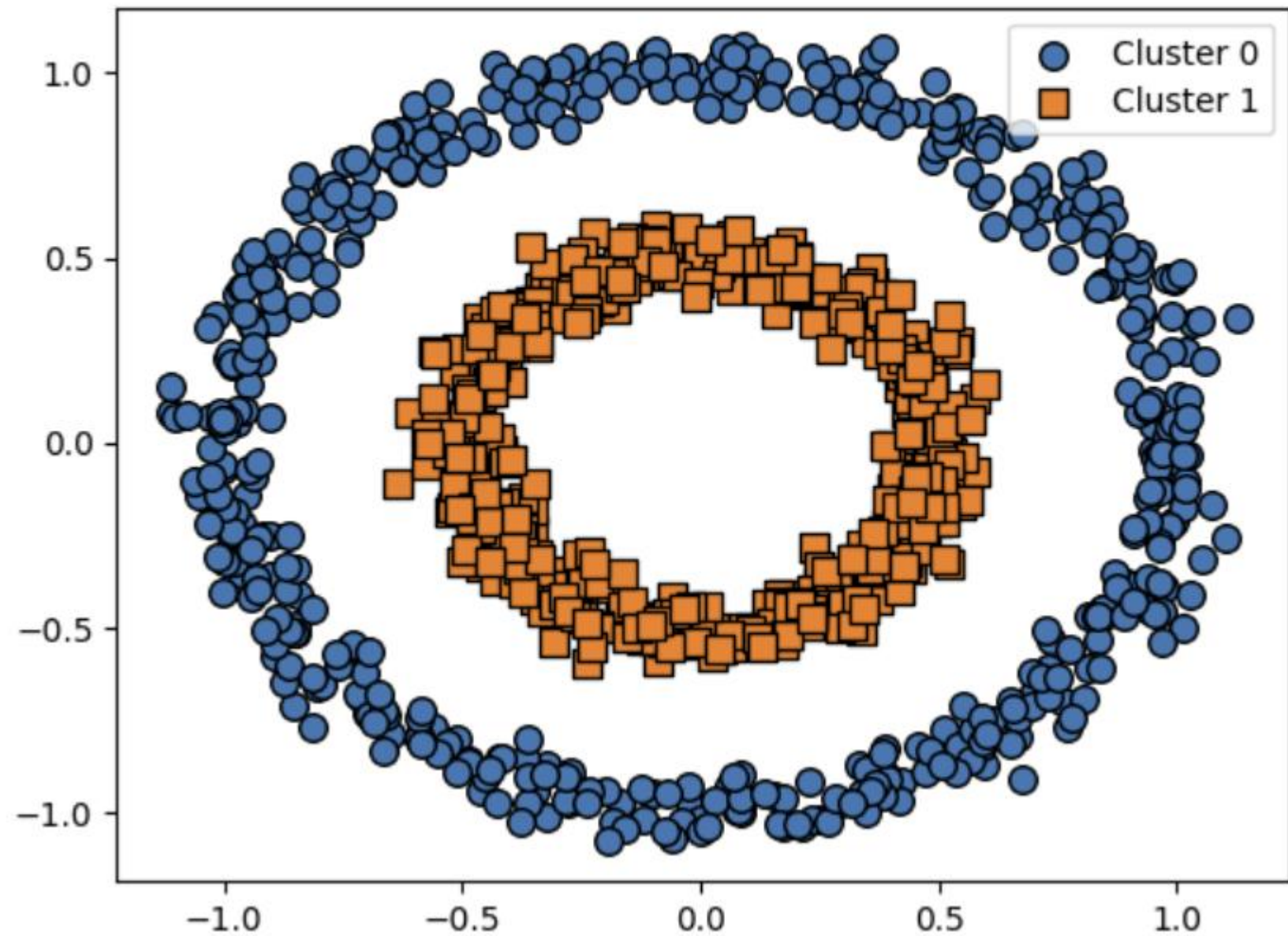
`factor`

: 외부 원, 내부 원 scale 비율

```
from sklearn.datasets import make_circles
```

```
X,y = make_circles(n_samples=1000, shuffle=True, noise=0.05, random_state=0, factor=0.5)  
clusterDF = pd.DataFrame(data=X, columns=['ftr1', 'ftr2'])  
clusterDF['target'] = y
```

```
visualize_cluster_plot(None, clusterDF, 'target', iscenter=False)
```





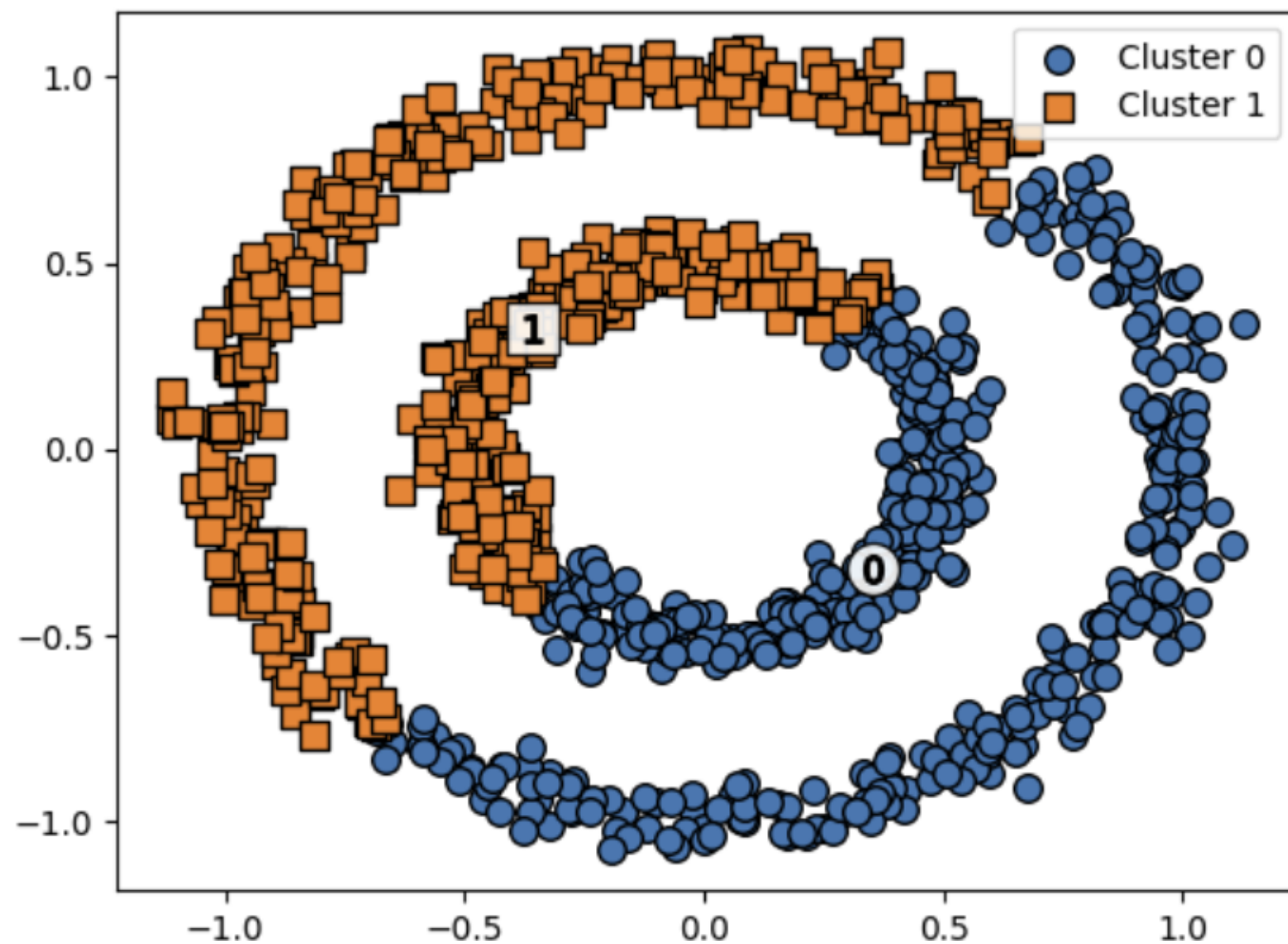
# 05 DBSCAN

## KMeans

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, max_iter = 1000, random_state=0)
kmeans_labels = kmeans.fit_predict(X)
clusterDF['kmeans_cluster'] = kmeans_labels

visualize_cluster_plot(kmeans, clusterDF, 'kmeans_cluster', iscenter=True)
```

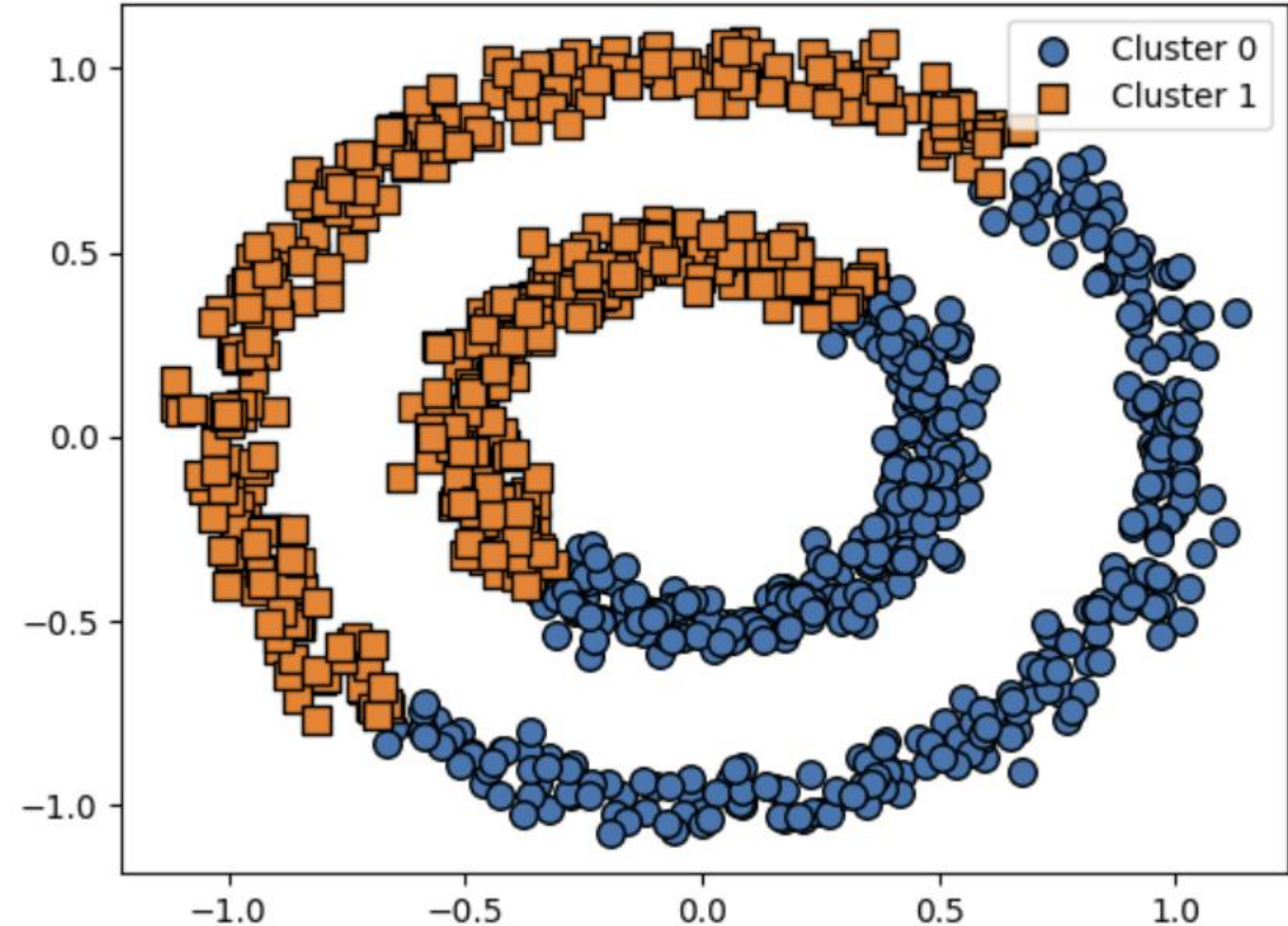


## GMM

```
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(n_components=2, random_state=0)
gmm_label = gmm.fit(X).predict(X)
clusterDF['gmm_cluster'] = gmm_label

visualize_cluster_plot(gmm, clusterDF, 'gmm_cluster', iscenter=False)
```

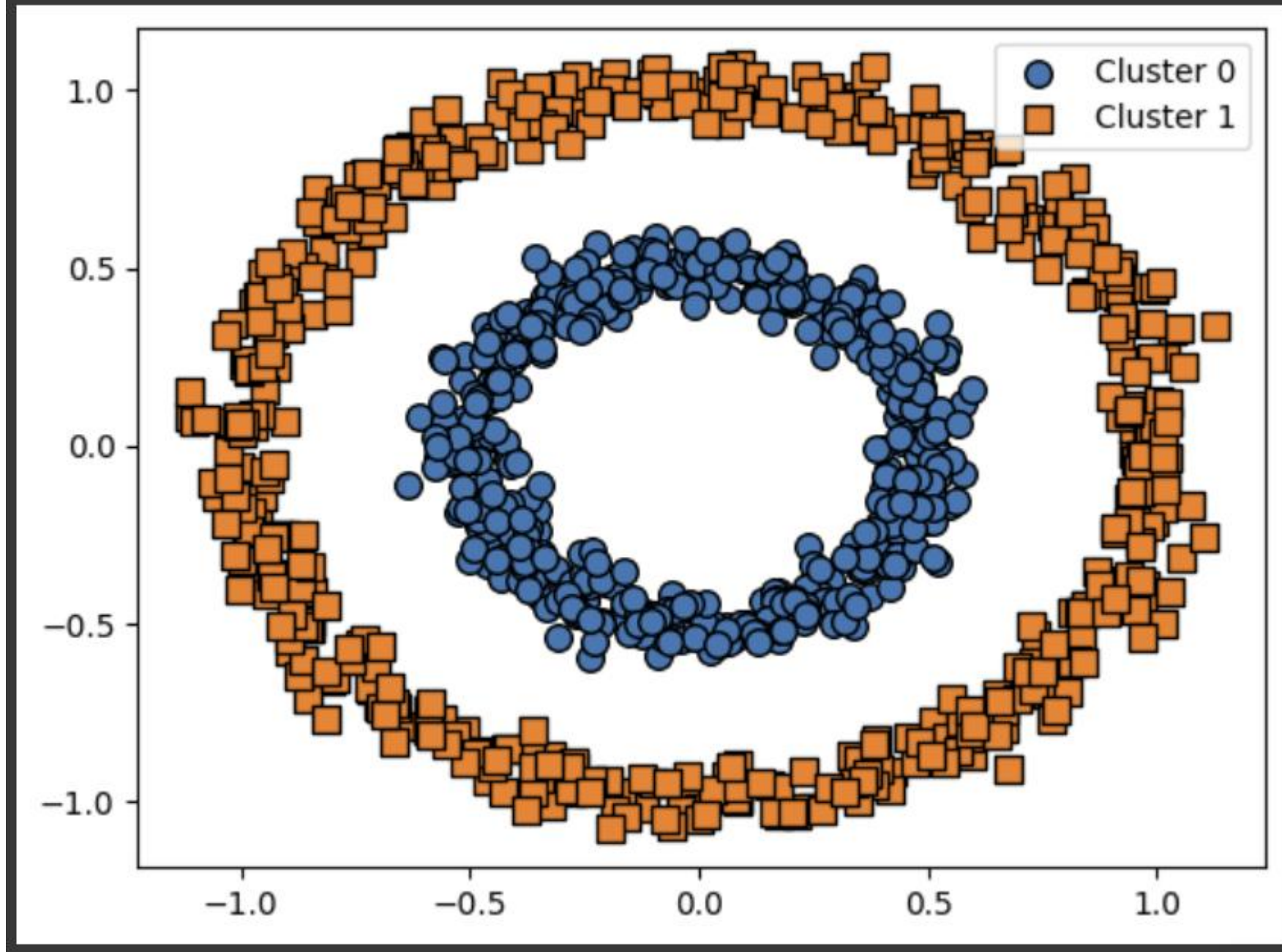


# 05 DBSCAN

```
from sklearn.cluster import DBSCAN

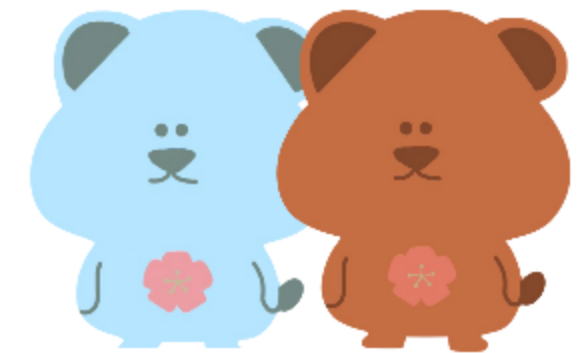
dbscan = DBSCAN(eps=0.2, min_samples=10, metric='euclidean')
dbscan_labels = dbscan.fit_predict(X)
clusterDF['dbscan_cluster'] = dbscan_labels

visualize_cluster_plot(dbscan, clusterDF, 'dbscan_cluster', iscenter=False)
```



## DBSCAN

+ 병합 군집 & 계층적 군집





# # 병합 군집

## 병합 군집(Agglomerative Clustering)

: 각 데이터 포인트를 하나의 클러스터로 지정

-> 지정된 개수의 클러스터가 남을 때까지 **가장 비슷한 두 클러스터를 합쳐 나가는 알고리즘**

### < Method >

- Ward: 모든 클러스터 내의 분산을 가장 작게 증가시키는 두 클러스터를 합치는 방식
- Average: 클러스터 포인트 사이의 평균 거리가 가장 짧은 두 클러스터를 합치는 방식
- Complete: 클러스터 포인트 사이의 최대 거리가 가장 짧은 두 클러스터를 합치는 방식
- Single: 클러스터 포인트 사이의 최소 거리가 가장 짧은 두 클러스터를 합치는 방식

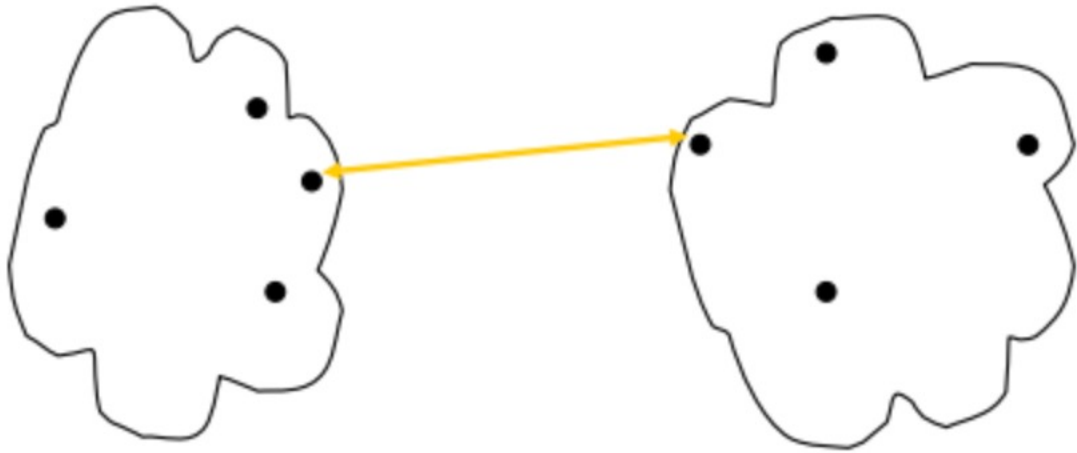


# # 병합 군집

Linkage 옵션 : 가장 비슷한 클러스터를 측정하는 방법을 제시

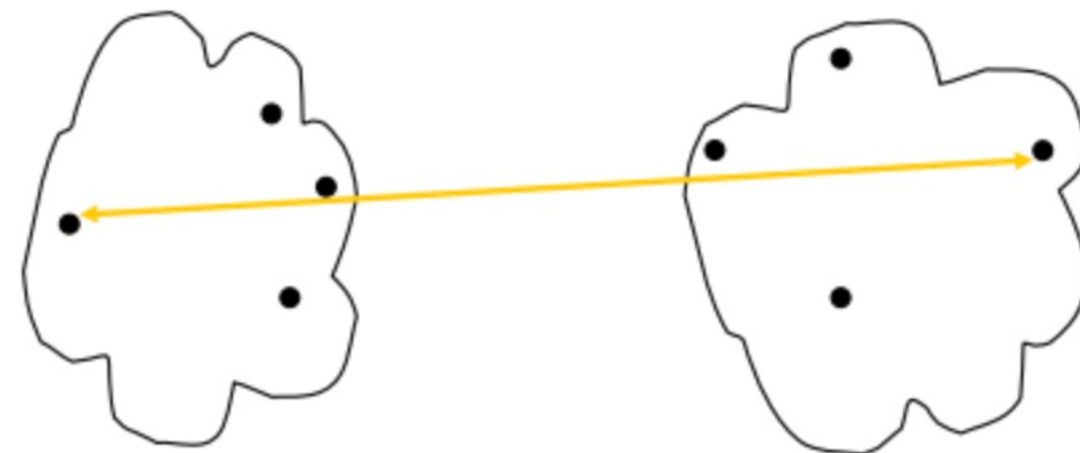
Single(Minimum)

: **최소 거리**가 가장 짧은 두 클러스터 병합



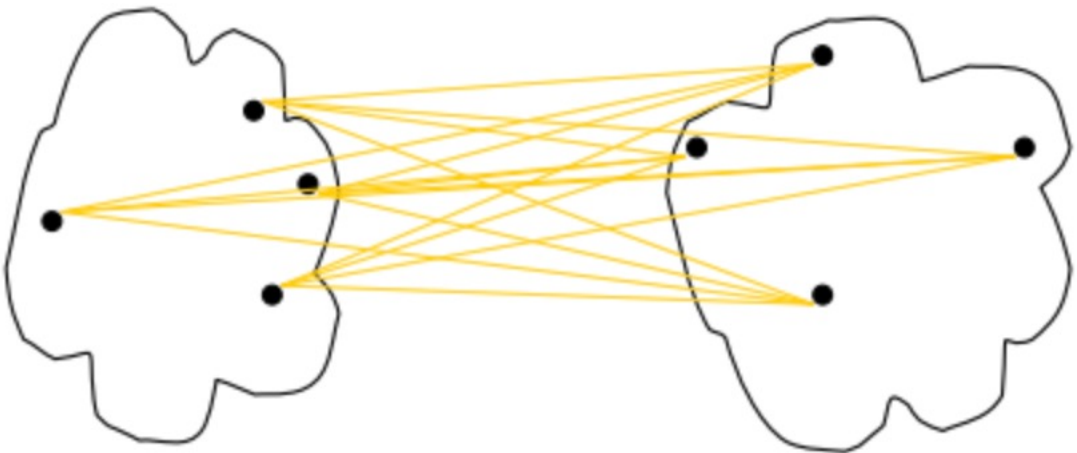
Complete (Maximum)

: **최대 거리**가 가장 짧은 두 클러스터 병합



Average

: **평균 거리**가 가장 짧은 두 클러스터 병합



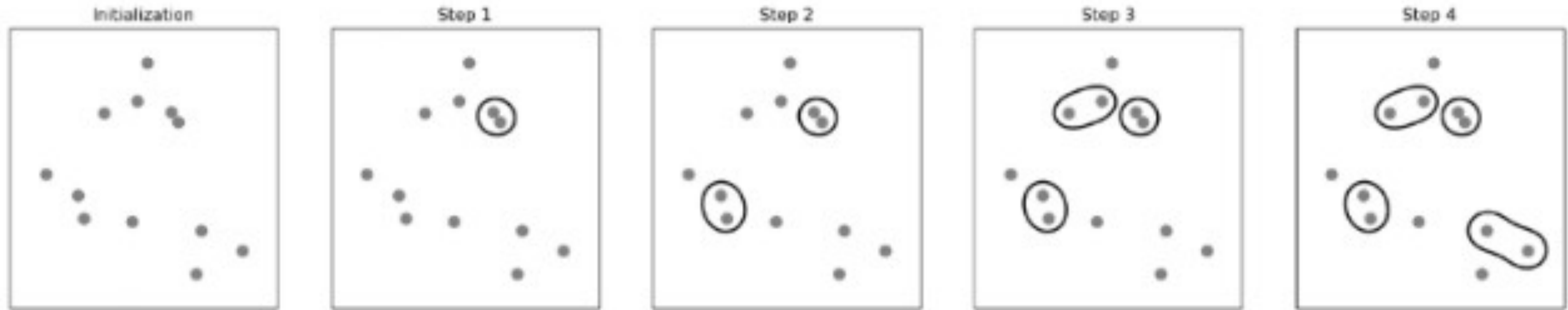
Ward

: **분산을 가장 작게** 증가시키는 두 클러스터 병합

- Noise, outlier에 덜 취약
- 크기가 비슷한 클러스터가 만들어짐
- 가장 잘 적용되는 방식 (default)

# # 병합 군집

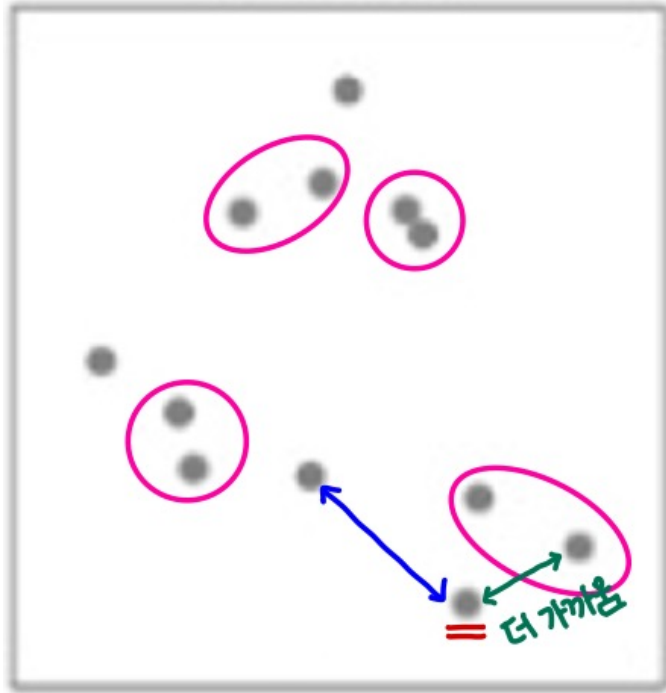
지정 클러스터 개수: 3



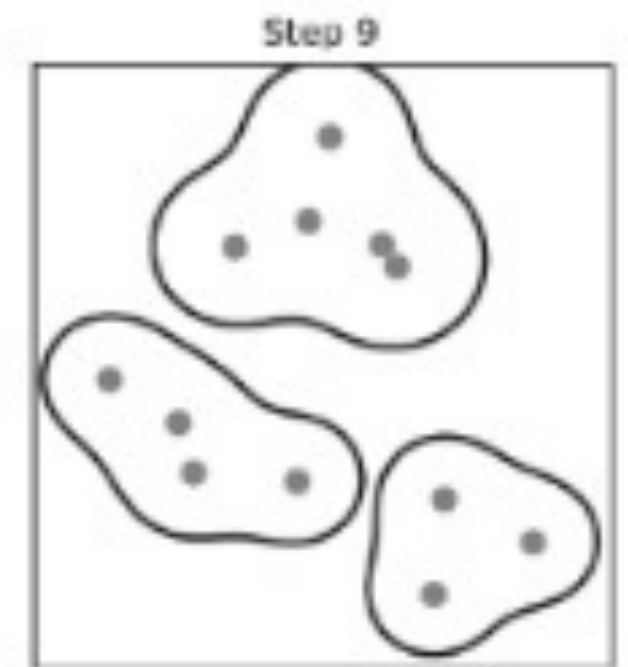
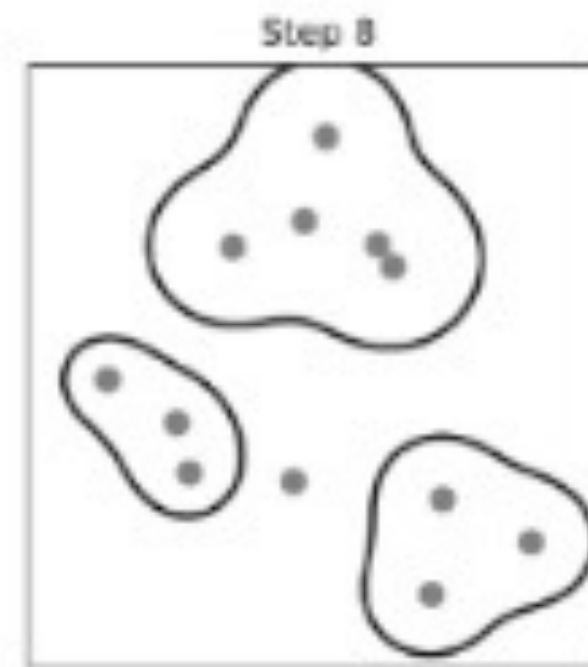
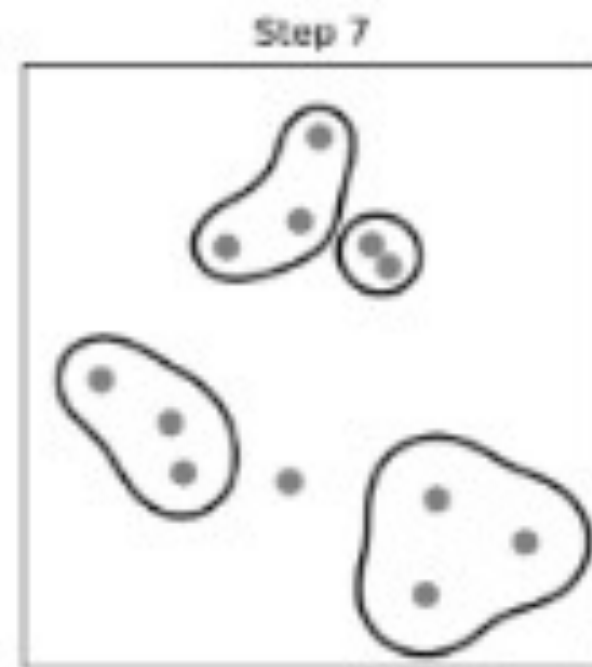
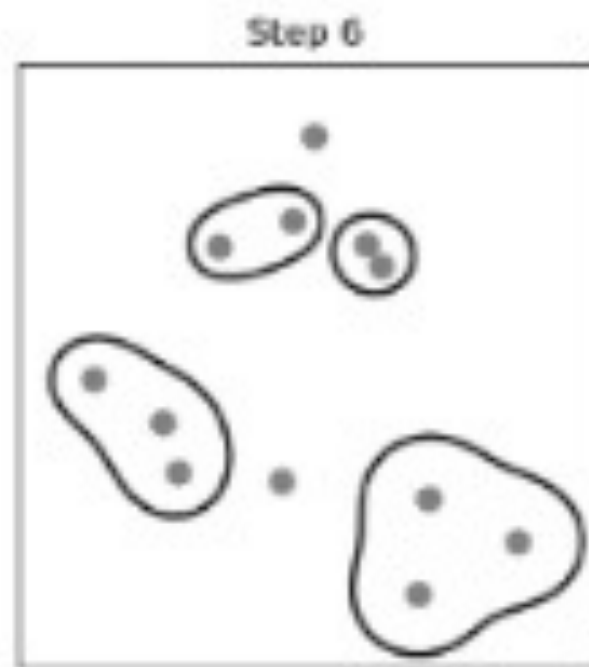
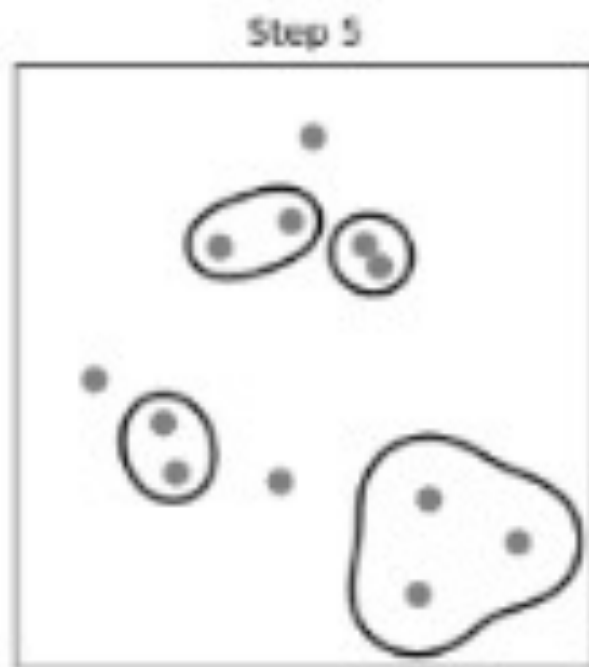
Init: 각 포인터가 하나의 클러스터

- ➔ 가장 가까운 두 클러스터가 합쳐짐 (반복)
- ➔ 점점 더 멀리 있는 데이터를 포함시킴
- ➔ 지정 클러스터 개수가 되면 병합을 멈춤

# # 병합 군집



- Init: 각 포인트가 하나의 클러스터
- ➔ 가장 가까운 두 클러스터가 합쳐짐 (반복)
  - ➔ 점점 더 멀리 있는 데이터를 포함시킴
  - ➔ 지정 클러스터 개수가 되면 병합을 멈춤



# # 계층적 군집

## 계층적 군집(Hierarchical Clustering)

: 계층적 트리 모형을 이용

- > 개별 데이터 포인트들을 **순차적, 계층적으로 유사한 클러스터**로 통합하여 군집화를 수행하는 알고리즘
- 클러스터의 개수를 사전에 정하지 않아도 학습을 수행할 수 있음

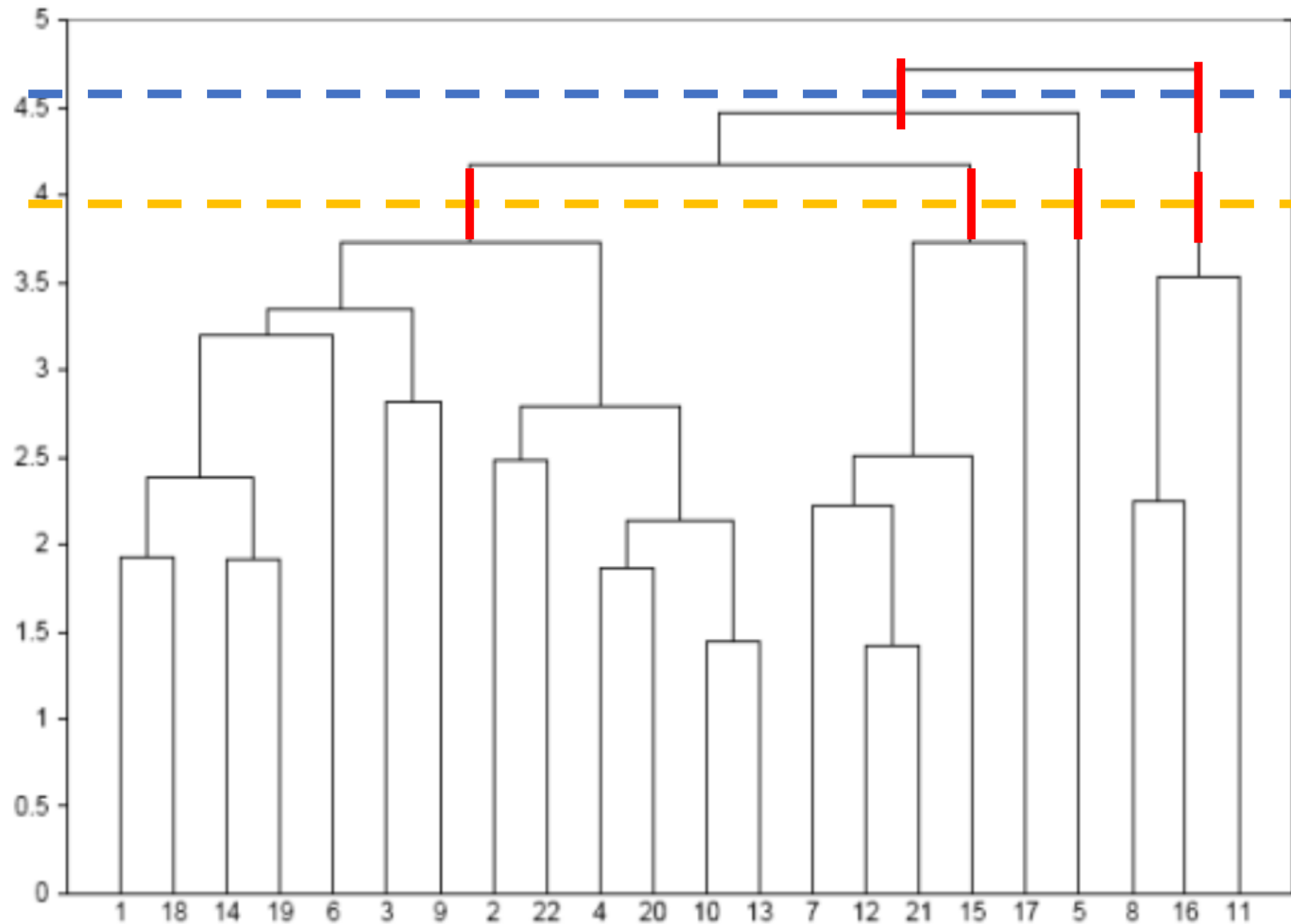
### < Method >

- 병합 군집 방식: 각 포인터를 하나의 클러스터로 지정 -> 순차적으로 병합 (상향식)
- 분할 군집 방식: 전체를 하나의 클러스터로 지정 -> 순차적으로 분할 (하향식)

### < 시각화 >

덴드로그램(Dendrogram): 개체들이 결합되는 순서를 나타내는 트리 형태의 구조

# # 계층적 군집



2개의 cluster

4개의 cluster

- 가지의 길이  
: 합쳐진 클러스터가 얼마나 멀리 떨어져 있는지
- 가로선 분할 -> 클러스터를 임의로 나눌 수 있음  
=> 사용자가 클러스터를 나눌 수 있음



## 7.6 군집화 실습



# # 고객 세그먼테이션의 정의와 기법

## 고객 세그먼테이션(Customer Segmentation)

: 다양한 기준으로 고객을 분류하는 기법

어떤 상품을 얼마나 많은 비용을 써서 얼마나 자주 사용하는가?  
=> 얼마나 많은 매출을 발생하느냐

### < 목표: 타겟 마케팅 >

: 고객을 여러 특성에 맞게 세분화 -> 맞춤형 마케팅 or 서비스 제공  
- Vip 선정, 프로모션 메일

“ 고객의 어떤 요소를 기반으로 군집화 할 것인가? ”

# # 고객 세그멘테이션의 정의와 기법

## RMF 기법

Recency(R): 가장 최근 상품 구입 일에서 오늘까지의 기간

Frequency(F): 상품 구매 횟수

Monetary Value(M): 총 구매 금액

# # 데이터 세트 로딩과 데이터 클렌징

제품 주문 데이터 세트 read\_excel( )

|   | InvoiceNo | StockCode | Description                        | Quantity | InvoiceDate         | UnitPrice | CustomerID | Country        |
|---|-----------|-----------|------------------------------------|----------|---------------------|-----------|------------|----------------|
| 0 | 536365    | 85123A    | WHITE HANGING HEART T-LIGHT HOLDER | 6        | 2010-12-01 08:26:00 | 2.55      | 17850.0    | United Kingdom |
| 1 | 536365    | 71053     | WHITE METAL LANTERN                | 6        | 2010-12-01 08:26:00 | 3.39      | 17850.0    | United Kingdom |
| 2 | 536365    | 84406B    | CREAM CUPID HEARTS COAT HANGER     | 8        | 2010-12-01 08:26:00 | 2.75      | 17850.0    | United Kingdom |

- InvoiceNo: 주문번호
- StockCode: 제품 코드
- Description: 제품 설명
- Quantity: 주문 제품 건수
- InvoiceDate: 주문 일자
- UnitPrice: 제품 단가
- CustomerID: 고객 번호
- Country: 국가명 (고객의 국적)

# # 데이터 세트 로딩과 데이터 클렌징

df.info(): 전체 데이터 수, 칼럼 타입, Null 개수 확인

```
retail_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        541909 non-null object
1   StockCode       541909 non-null object
2   Description     540455 non-null object
3   Quantity        541909 non-null int64
4   InvoiceDate      541909 non-null datetime64[ns]
5   UnitPrice       541909 non-null float64
6   CustomerID      406829 non-null float64
7   Country         541909 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB
```

< 전처리 >

Null 데이터 제거  
: CustomerID의 null 값이 13만 5천건

오류 데이터 삭제  
: Quantity, UnitPrice가 0보다 작은 경우



# # 데이터 세트 로딩과 데이터 클렌징

사전 정제 작업 후 null 값 재확인

불린 인덱싱: Quantity > 0, UnitPrice > 0  
CustomerID의 null값 제거

```
retail_df=retail_df[retail_df['Quantity']>0]
retail_df=retail_df[retail_df['UnitPrice']>0]
retail_df=retail_df[retail_df['CustomerID'].notnull()]
print(retail_df.shape)
retail_df.isnull().sum().to_frame().T
```

(397884, 8)

|   | InvoiceNo | StockCode | Description | Quantity | InvoiceDate | UnitPrice | CustomerID | Country |
|---|-----------|-----------|-------------|----------|-------------|-----------|------------|---------|
| 0 | 0         | 0         | 0           | 0        | 0           | 0         | 0          | 0       |

# # 데이터 세트 로딩과 데이터 클렌징

Country: 영국이 대다수 차지 -> 나머지 데이터 제외

```
retail_df['Country'].value_counts()[ :5].to_frame().T
```

| Country | United Kingdom | Germany | France | EIRE | Spain |
|---------|----------------|---------|--------|------|-------|
| count   | 354321         | 9040    | 8341   | 7236 | 2484  |



```
retail_df=retail_df[retail_df['Country']=='United Kingdom']  
print(retail_df.shape)
```

```
(354321, 8)
```

# # RFM 기반 데이터 가공

주문 금액 데이터:  $\text{UnitPrice} * \text{Quantity}$   
CustomerNo: float  $\rightarrow$  int

```
retail_df['sale_amount']=retail_df['Quantity']*retail_df['UnitPrice']  
retail_df['CustomerID']=retail_df['CustomerID'].astype(int)
```

# # RFM 기반 데이터 가공

Top5 주문 건수 & 주문 금액을 가진 고객 추출

개인 고객 + 소매점 주문 -> 특정 고객이 많은 주문 건수와 주문 금액을 가짐

```
print(etail_df['CustomerID'].value_counts().head(5))  
print(etail_df.groupby('CustomerID')['sale_amount'].sum().sort_values(ascending=False)[:5])
```

CustomerID

17841      7847

14096      5111

12748      4595

14606      2700

15311      2379

Name: count, dtype: int64

CustomerID

18102      259657.30

17450      194550.79

16446      168472.50

17511      91062.38

16029      81024.84

Name: sale\_amount, dtype: float64

# # RFM 기반 데이터 가공

Group by InvoiceNo + StockCode

```
retail_df.groupby(['InvoiceNo', 'StockCode'])['InvoiceNo'].count().mean()  
  
np.float64(1.028702077315023)
```

대부분의 그룹이 1개의 행만 가짐  
⇒ 거의 하나의 주문번호마다 하나의 식별자만 대응됨

\*\* 목적: **고객 레벨**로 주문 기간, 주문 횟수, 주문 금액 데이터를 기반으로 세그먼테이션 수행  
주문번호 + 상품코드 기준의 데이터 -> **고객 기준의 R,F,M 데이터**로 변경



# # RFM 기반 데이터 가공

Group by CustomerID -> 새 DF 생성

agg( ): 칼럼 여러 개에 서로 다른 aggregation 연산 수행

```
aggregations={'InvoiceDate':'max','InvoiceNo':'count','sale_amount':'sum'}
cust_df=retail_df.groupby('CustomerID').agg(aggregations)
cust_df=cust_df.rename(columns={'InvoiceDate':'Recency','InvoiceNo':'Frequency','sale_amount':'Monetary'})
cust_df=cust_df.reset_index()
cust_df.head(3)
```

|   | CustomerID | Recency             | Frequency | Monetary |
|---|------------|---------------------|-----------|----------|
| 0 | 12346      | 2011-01-18 10:01:00 | 1         | 77183.60 |
| 1 | 12747      | 2011-12-07 14:34:00 | 103       | 4196.01  |
| 2 | 12748      | 2011-12-09 12:20:00 | 4595      | 33719.73 |



# # RFM 기반 데이터 가공

Recency: 오늘 날짜 – 가장 최근 주문 일자

```
import datetime as dt
cust_df['Recency']=dt.datetime(2011,12,10)-cust_df['Recency']
cust_df['Recency']=cust_df['Recency'].apply(lambda x:x.days+1)
print('cust_df 로우와 칼럼 건수는',cust_df.shape)
cust_df.head(3)
```

cust\_df 로우와 칼럼 건수는 (3920, 4)

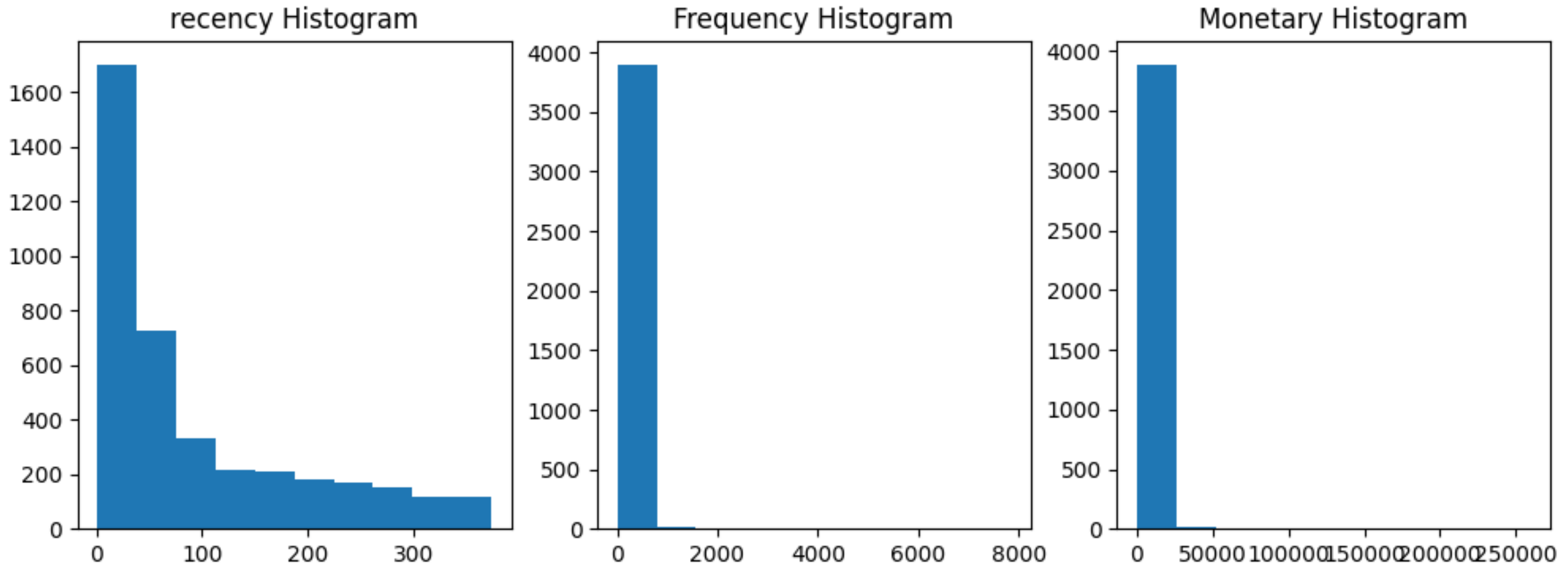
|   | CustomerID | Recency | Frequency | Monetary |
|---|------------|---------|-----------|----------|
| 0 | 12346      | 326     | 1         | 77183.60 |
| 1 | 12747      | 3       | 103       | 4196.01  |
| 2 | 12748      | 1       | 4595      | 33719.73 |



# # RFM 기반 고객 세그먼테이션

왜곡된 데이터: 군집화가 한쪽 군집에 집중되는 현상 발생

hist( ): 각 칼럼의 값 분포도 확인



# # RFM 기반 고객 세그먼테이션

describe( ): 각 칼럼의 데이터 값 백분위 확인

|       | Recency     | Frequency   | Monetary      |
|-------|-------------|-------------|---------------|
| count | 3920.000000 | 3920.000000 | 3920.000000   |
| mean  | 92.742092   | 90.388010   | 1864.385601   |
| std   | 99.533485   | 217.808385  | 7482.817477   |
| min   | 1.000000    | 1.000000    | 3.750000      |
| 25%   | 18.000000   | 17.000000   | 300.280000    |
| 50%   | 51.000000   | 41.000000   | 652.280000    |
| 75%   | 143.000000  | 99.250000   | 1576.585000   |
| max   | 374.000000  | 7847.000000 | 259657.300000 |

# # RFM 기반 고객 세그먼테이션

StandardScaler(): 평균과 표준편차 재조정

-> K-평균 수행

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, silhouette_samples

X_features=cust_df[['Recency', 'Frequency', 'Monetary']].values
X_features_scaled=StandardScaler().fit_transform(X_features)

kmeans=KMeans(n_clusters=3, random_state=0, n_init=10)
labels=kmeans.fit_predict(X_features_scaled)
cust_df['cluster_label']=labels

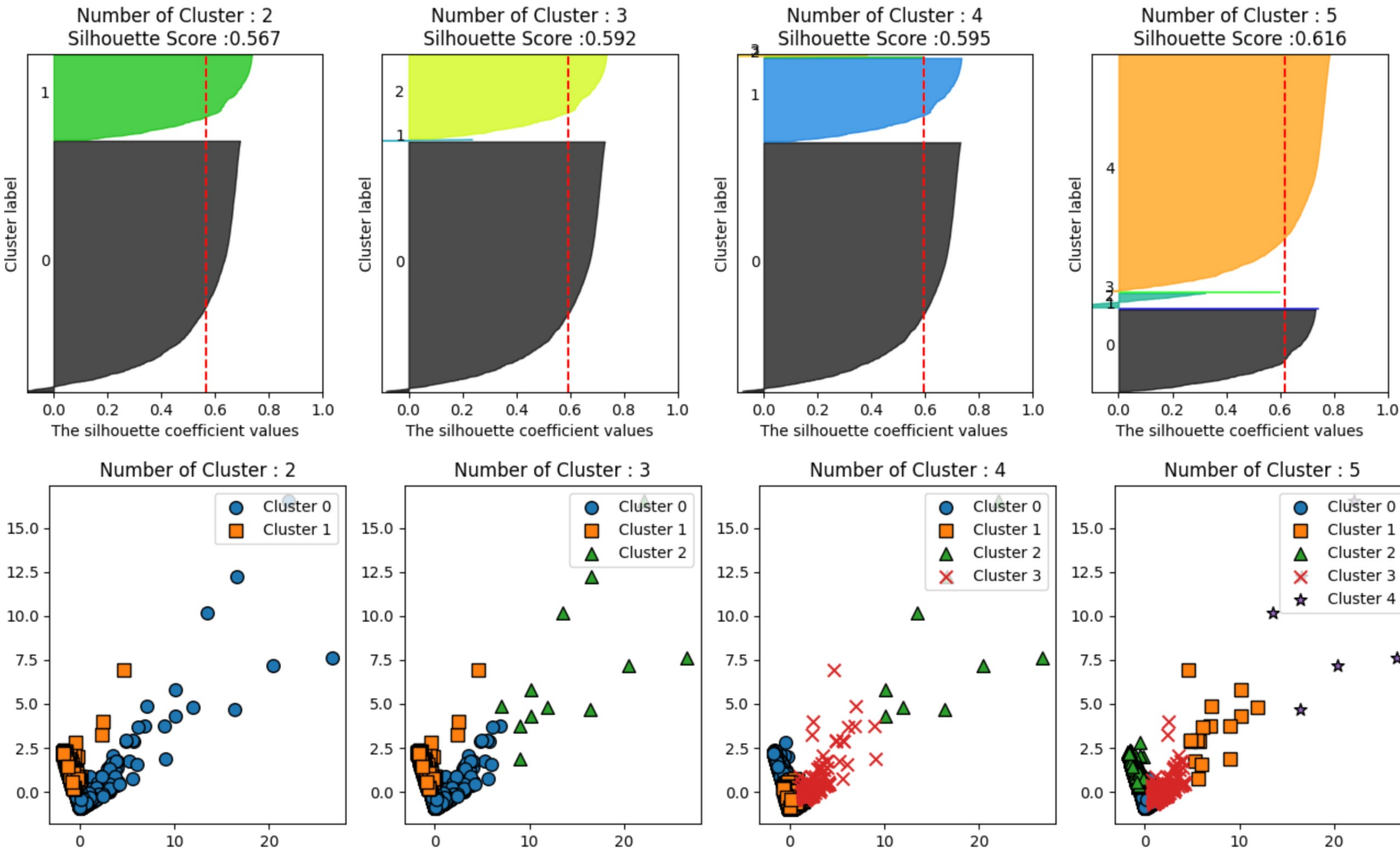
print('실루엣 스코어 : {0:.3f}'.format(silhouette_score(X_features_scaled, labels)))
```

실루엣 스코어 : 0.592



# # RFM 기반 고객 세그먼테이션

## 개별 군집의 실루엣 계수 & 데이터 구성 시각화



군집 수를 증가시키면  
데이터 세트의 수가  
너무 작은 군집 생성

군집 내부: 데이터가  
광범위하게 퍼짐

=> 의미 없는 군집화 반복



# # RFM 기반 고객 세그먼테이션

Log 변환: 왜곡 정도를 낮추기 위해 사용

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, silhouette_samples

cust_df['Recency_log'] = np.log1p(cust_df['Recency'])
cust_df['Frequency_log'] = np.log1p(cust_df['Frequency'])
cust_df['Monetary_log'] = np.log1p(cust_df['Monetary'])

X_features = cust_df[['Recency_log', 'Frequency_log', 'Monetary_log']].values
X_features_scaled = StandardScaler().fit_transform(X_features)

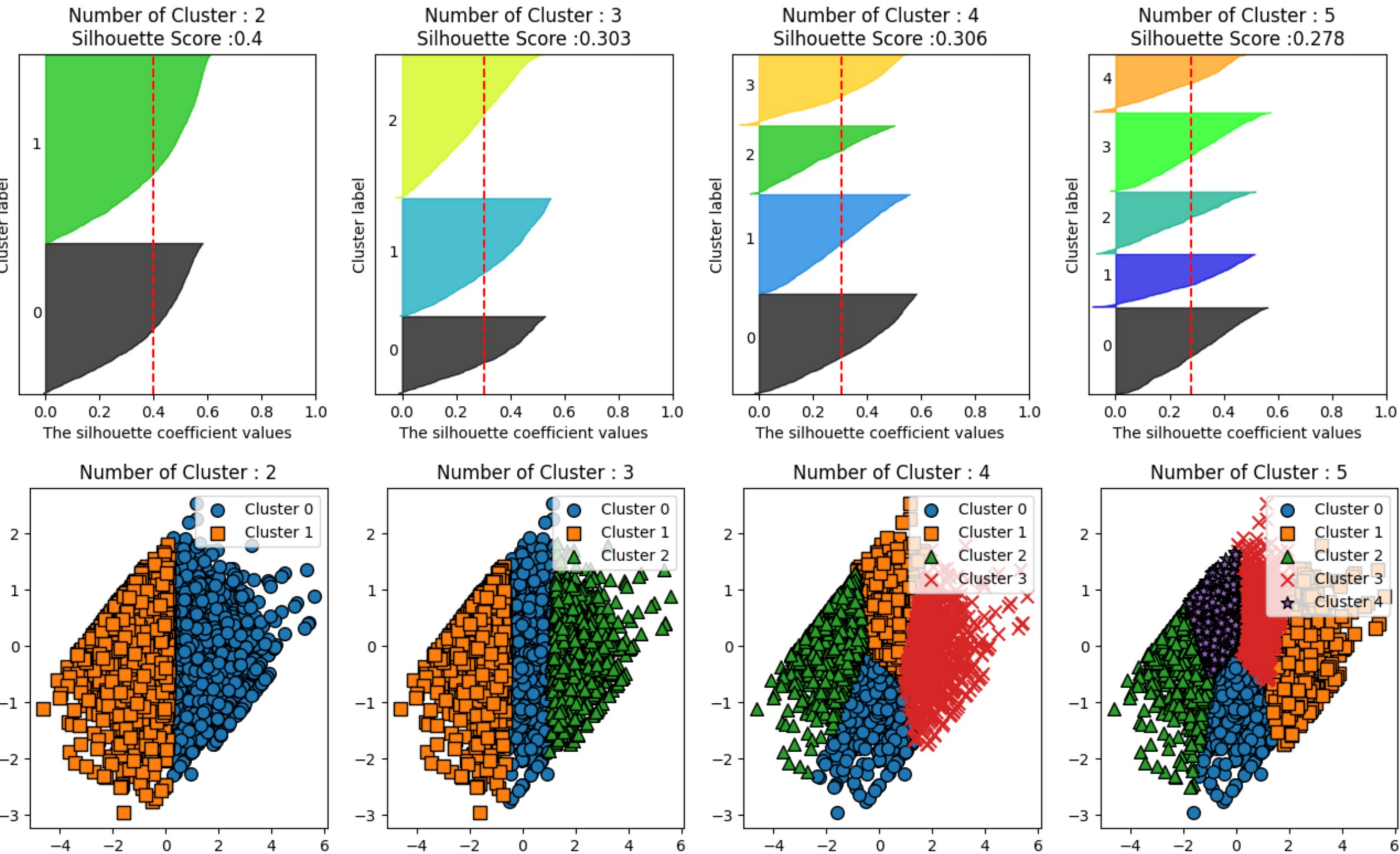
kmeans = KMeans(n_clusters=3, random_state=0)
labels = kmeans.fit_predict(X_features_scaled)
cust_df['cluster_label'] = labels

print('실루엣 스코어 : {0:.3f}'.format(silhouette_score(X_features_scaled, labels)))
```

실루엣 스코어 : 0.303

# # RFM 기반 고객 세그먼테이션

## Log 변환 후 실루엣 계수 & 군집화 구성 시각화



실루엣 스코어: 감소  
But 더 균일한  
군집화 구성

# THANK YOU

